



UNIVERSIDADE DE ÉVORA
ESCOLA DE CIÊNCIAS E TECNOLOGIA

DEPARTAMENTO DE INFORMÁTICA

Exploração e Teste de Técnicas de AutoML

Licenciatura em Inteligência Artificial e Ciências de Dados
Estágio-Projeto 2025/2026



Ana Patrícia Gil

Orientador na Empresa: Nuno Miranda

Orientador no Departamento: Teresa Gonçalves

Trabalho desenvolvido na empresa KPMG, no âmbito da disciplina de Estágio-Projeto da Licenciatura em Inteligência Artificial e Ciências de Dados.

Évora, 19 de junho de 2026

Resumo

Este trabalho, desenvolvido na KPMG Portugal, teve como objetivo avaliar e comparar sete ferramentas de AutoML: TPOT, AutoGluon, H2O AutoML e AdaNet, no segmento open-source, e Azure ML AutoML, AWS SageMaker Canvas e Google Vertex AI AutoML, no segmento cloud. A avaliação recorreu aos conjuntos de dados "Iris" e "Breast Cancer Wisconsin".

Em problemas simples, todas as ferramentas apresentaram desempenhos semelhantes, sendo as diferenças mais evidentes em dados de maior complexidade. Entre as soluções open-source, o AutoGluon e o H2O AutoML destacaram-se no desempenho preditivo, o TPOT pela exportabilidade do *pipeline* como código Python e o AdaNet mostrou-se mais adequado a contextos de investigação. Nas plataformas cloud, o SageMaker Canvas revelou-se a opção mais ágil para prototipagem rápida, o Azure ML a mais indicada para projetos que exigem auditabilidade e o Vertex AI a mais adequada para organizações já integradas no ecossistema Google Cloud.

A principal conclusão é que nenhuma ferramenta se impõe como universalmente superior, a escolha deve ser orientada pela natureza do problema, pelas características dos dados e pelos requisitos operacionais de cada projeto.

Agradecimentos

À **Professora Teresa Gonçalves**, do Departamento de Informática da Universidade de Évora, agradeço a orientação académica e a disponibilidade demonstrada ao longo de todo o estágio, cujo acompanhamento foi fundamental para a concretização deste trabalho.

Ao **Miguel Silvério**, Data Scientist na KPMG Portugal, um agradecimento especial pela orientação técnica diária, pela partilha de conhecimento e pelo apoio constante no desenvolvimento do projeto. A sua disponibilidade e proximidade foram determinantes para ultrapassar os desafios encontrados ao longo do estágio.

Utilização de Inteligência Artificial

No desenvolvimento deste trabalho recorri a ferramentas de Inteligência Artificial Generativa para apoio à revisão ortográfica e melhoria da clareza expositiva. Todo o conteúdo produzido ou assistido por Inteligência Artificial foi revisto, validado e editado por mim, pelo que assumo plena responsabilidade pelo rigor, originalidade, transparência e integridade científica do trabalho que aqui se materializa. Considero-me, neste sentido, plena Autora do texto submetido.

GitHub - [anapatriciagil74-arch/projeto-estagio](https://github.com/anapatriciagil74-arch/projeto-estagio)

Repositório pessoal onde se encontram todos os ficheiros de código desenvolvidos neste trabalho e pastas exportadas das plataformas cloud.

Tabela de Conteúdos

1 Introdução	1
1.1 Enquadramento Institucional	2
1.2 Objetivos	2
1.3 Contribuições	3
1.4 Estrutura do Documento	3
2 Ambiente de Trabalho	5
2.1 Estrutura Organizacional	5
2.2 Equipa de Data & Analytics	5
2.3 Orientação e Acompanhamento	5
2.4 Metodologia de Trabalho e Colaboração	6
2.5 Inserção no Projeto e Contribuição para Projetos Paralelos	6
3 Fundamentação Teórica	7
3.1 Conceito de AutoML	7
3.2 Funcionamento do AutoML	7
3.3 Aplicações do AutoML	8
3.4 Vantagens e Desvantagens do AutoML	8
3.5 Estado da Arte	8
4 Contexto Operacional	10
4.1 Ambiente Tecnológico	10
4.1.1 Hardware	10
4.1.2 Software	10
4.1.3 Ambiente aplicacional	11
4.2 Metodologia de trabalho	11
4.2.1 Processos de Desenvolvimento de Software	11
4.2.2 Gestão do Processo de Desenvolvimento	12
4.2.3 Planificação do Trabalho	12
4.2.4 Interação com os Restantes Membros da Equipa	13
5 Trabalho desenvolvido	14
5.1 Formação Inicial e Levantamento Teórico	15
5.2 Exploração Prática das Ferramentas Open-Source	16
5.2.1 TPOT	17
5.2.1.1 Configuração	17
5.2.1.2 Exploração Prática e Análise Crítica - “Iris”	17

5.2.1.3 Exploração Prática e Análise Crítica - "Breast Cancer Wisconsin"	18
5.2.1.4 Conclusões	18
5.2.2 AutoGluon	19
5.2.2.1 Configuração	19
5.2.2.2 Exploração Prática e Análise Crítica - "Iris"	19
5.2.2.3 Exploração Prática e Análise Crítica - "Breast Cancer Wisconsin"	20
5.2.2.4 Conclusões	20
5.2.3 H2O AutoML	21
5.2.3.1 Configuração	21
5.2.3.2 Exploração Prática e Análise Crítica - "Iris"	21
5.2.3.3 Exploração Prática e Análise Crítica - "Breast Cancer Wisconsin"	21
5.2.3.4 Conclusões	22
5.2.4 AdaNet	22
5.2.4.1 Configuração	23
5.2.4.2 Exploração Prática e Análise Crítica - "Iris"	23
5.2.4.3 Exploração Prática e Análise Crítica - "Breast Cancer Wisconsin"	23
5.2.4.4 Conclusões	24
5.3 Análise Comparativa das Ferramentas Open-Source	24
5.3.1 "Iris"- Isolamento do Efeito da Ferramenta	24
5.3.2 "Breast Cancer Wisconsin" - Emergência das Diferenças Reais	25
5.3.3 Resultados Comparativos	25
5.3.4 Síntese Crítica, SWOT Comparativa e Recomendações de Uso	26
5.4 Exploração Prática das Plataformas Cloud	29
5.4.1 Azure ML AutoML	29
5.4.1.1 Conjunto de Dados e Importação	29
5.4.1.2 Infraestrutura e Computação	30
5.4.1.3 Resultados do Modelo	31
5.4.1.3.1 Modelo Vencedor e Ensemble Details	32
5.4.1.4 Tabela de Classificação	33
5.4.1.5 Outputs, Logs e Experiment View	33
5.4.1.6 Conclusões	34
5.4.2 AWS SageMaker Canvas	34
5.4.2.1 Conjunto de Dados e Importação	35
5.4.2.2 Configuração do Modelo	35
5.4.2.3 Resultados do Modelo	36
5.4.2.3.1 Importância dos Atributos (SHAP)	36

5.4.2.3.2 Scoring e Matriz de Confusão	36
5.4.2.3.3 Métricas Avançadas e Curva Precisão-Recall	37
5.4.2.4 Tabela de Classificação	38
5.4.2.5 Previsões e Deploy	38
5.4.2.6 Conclusões	39
5.4.3 Google Vertex AI AutoML	39
5.4.3.1 Conjunto de Dados e Importação	40
5.4.3.1.1 Primeira Tentativa – “Breast Cancer Wisconsin”	40
5.4.3.1.2 Segunda Tentativa – “Adult Income”	40
5.4.3.2 Configuração do AutoML Job	41
5.4.3.3 Resultados do Modelo	42
5.4.3.3.1 Curvas de Avaliação e Matriz de Confusão	43
5.4.3.3.2 Importância dos Atributos (SHAP)	44
5.4.3.4 Tabela de Classificação e Transparência do Processo	44
5.4.3.5 Deploy e Integração	44
5.4.3.6 Conclusões	45
5.5 Análise Comparativa das Plataformas Cloud	45
5.5.1 Implicações para a Prática de Consultoria	46
5.5.2 Síntese Crítica e SWOT Comparativa	47
5.6 Comparação entre Ferramentas Open-Source e Plataformas Cloud	49
5.6.1 Acessibilidade e Curva de Aprendizagem	49
5.6.2 Controle, Transparência e Auditabilidade	49
5.6.3 Desempenho Preditivo e Qualidade dos Modelos	50
5.6.4 Custo e Escalabilidade	50
5.6.5 Síntese Comparativa	51
6 Avaliação Crítica	52
6.1 Aprendizagens Técnicas	52
6.2 Lições Metodológicas	52
6.3 Dimensão Humana e Comunicativa	53
6.4 Propostas para Trabalhos Futuros	53
6.5 Reflexões Finais	54
Glossário	55
Referências	56

1 Introdução

A adoção crescente de soluções de Inteligência Artificial nas empresas tem transformado a forma como estas abordam a análise de dados e a tomada de decisões. Neste contexto, o conceito de *Automated Machine Learning* (AutoML) surge como uma resposta à necessidade de democratizar o acesso a modelos de Aprendizagem Automática de qualidade, reduzindo a dependência de especialistas e acelerando o ciclo de desenvolvimento. Contudo, a proliferação de ferramentas e plataformas AutoML, desde soluções open-source até plataformas cloud de grande escala, criou um panorama complexo onde a seleção da ferramenta mais adequada a cada contexto é, por si só, um desafio.

As organizações enfrentam compromissos entre desempenho, custo, interpretabilidade e facilidade de uso, e a discrepância frequente entre resultados teóricos e desempenho real dificulta decisões informadas. Esta tendência é particularmente relevante em empresas de consultoria como a KPMG, que necessitam de implementar soluções de Aprendizagem Automática de forma rápida e escalável para os seus clientes.

O presente trabalho avalia de forma prática e comparativa ferramentas e plataformas de AutoML, divididas em dois grupos:

Ferramentas open-source:

- TPOT;
- AutoGluon;
- H2O AutoML;
- AdaNet.

Plataformas cloud:

- Azure ML AutoML;
- AWS SageMaker Canvas;
- Google Vertex AI AutoML.

Os resultados desta investigação mostram que nenhuma ferramenta domina universalmente todos os cenários, o que reforça a necessidade de avaliação contextualizada e sugere que a escolha da ferramenta de AutoML deve ser guiada pela

natureza do problema, pelas características dos dados e pelos requisitos operacionais específicos de cada projeto.

1.1 Enquadramento institucional

O estágio foi realizado na KPMG Portugal, no departamento de Tech Consulting, mais especificamente na equipa de Data & Analytics. Esta equipa dedica-se ao desenvolvimento de soluções nas áreas de Gen AI, Inteligência Artificial tradicional e análise de dados, atuando tanto em projetos internos de investigação e desenvolvimento como em projetos orientados para clientes.

O projeto surgiu da necessidade de documentar e comparar sistematicamente diferentes ferramentas de AutoML, de modo a apoiar futuras decisões de seleção tecnológica em projetos empresariais. Durante o período de estágio, a equipa de Data & Analytics integrava aproximadamente 100 profissionais com diferentes níveis de experiência e especialização, o que proporcionou um ambiente favorável à aprendizagem técnica, à partilha de conhecimento e ao contacto com práticas reais de consultoria tecnológica.

1.2 Objetivos

O objetivo principal do estágio consistiu na realização de uma exploração comparativa entre diferentes ferramentas e plataformas de AutoML. Mais especificamente, pretendeu-se:

- Verificar as funções e características de cada ferramenta AutoML, tanto open-source como cloud;
- Avaliar a facilidade de instalação, configuração e utilização de cada ferramenta em ambiente Windows;
- Comparar o funcionamento das ferramentas open-source aplicadas a conjuntos de dados de diferentes graus de complexidade ("Iris" e "Breast Cancer Wisconsin");
- Analisar os mecanismos de configuração de cada ferramenta, incluindo a possibilidade de utilização de ficheiros externos;
- Explorar e avaliar as plataformas cloud de AutoML, documentando o processo de configuração, os resultados obtidos e as limitações encontradas;

-
- Produzir documentação estruturada que sirva de recurso de referência interno para a equipa de Data & Analytics da KPMG na seleção de ferramentas AutoML para futuros projetos.

1.3 Contribuições

As principais contribuições realizadas durante o estágio incluíram:

- **Desenvolvimento de uma pseudo-framework de avaliação:** Criação de um conjunto de scripts Jupyter Notebook, através da plataforma Anaconda, comum a todas as ferramentas open-source, garantindo que todas as ferramentas estudadas foram executadas nas mesmas condições e de forma analiticamente consistente e reproduzível. Para as plataformas cloud, o elemento comum foram os mesmos conjuntos de dados utilizados nas ferramentas open-source, assegurando a compatibilidade dos resultados;
- **Análise comparativa detalhada:** Avaliação teórica e prática das ferramentas e plataformas de AutoML aplicadas a conjuntos de dados reais, com documentação de vantagens, desvantagens e recomendações de utilização;
- **Identificação de limitações de compatibilidade:** Identificação das restrições reais das ferramentas open-source em ambientes Windows, o que constitui um requisito da análise comparativa e informa decisões de infraestrutura para projetos futuros;
- **Documentação técnica:** Criação de relatórios detalhados com métricas de desempenho, análise SWOT comparativa e recomendações para diferentes casos de uso;
- **Validação da hipótese de adequação contextual:** Demonstração de que não existe uma ferramenta AutoML universalmente superior, a escolha ótima depende das características do problema, dos dados e dos requisitos operacionais específicos.

1.4 Estrutura do Documento

Este relatório está organizado da seguinte forma:

- **Capítulo 1 - Introdução:** Contextualização do estágio, apresentação dos objetivos e síntese das contribuições realizadas;

-
- **Capítulo 2 - Ambiente Empresarial:** Descrição da KPMG como organização, estrutura hierárquica, caracterização da equipa de Data & Analytics e metodologia de trabalho adotada;
 - **Capítulo 3 - Fundamentação Teórica:** Levantamento formal do conceito de AutoML e identificação das principais ferramentas e plataformas disponíveis;
 - **Capítulo 4 - Ambiente de Desenvolvimento:** Descrição do ambiente técnico utilizado (hardware, software, ambiente aplicacional), incluindo a infraestrutura de rede e as ferramentas de desenvolvimento, e metodologia de trabalho adotada;
 - **Capítulo 5 - Trabalho Desenvolvido:** Apresentação detalhada da exploração prática das ferramentas open-source e plataformas cloud de AutoML, estudo dos mecanismos de configuração e análise comparativa dos resultados;
 - **Capítulo 6 - Avaliação Crítica:** Reflexão sobre a experiência de estágio, aprendizagens técnicas e metodológicas, dimensão humana e comunicativa e por fim reflexões finais sobre o impacto do trabalho.

2 Ambiente de Trabalho

A KPMG é uma das “*Big Four*” empresas de auditoria e consultoria a nível mundial, operando em mais de 140 países e territórios. Em Portugal, estabeleceu-se como uma referência no mercado de consultoria, distinguindo-se pela sua capacidade de combinar experiência e conhecimento local com competência global.

2.1 Estrutura Organizacional

A KPMG Portugal está organizada em diferentes áreas de negócio, sendo o estágio realizado no departamento de Tech Consulting, mais especificamente na equipa de Data & Analytics. A estrutura hierárquica da área onde o estágio foi desenvolvido organiza-se da seguinte forma:

- **CEO (Lisboa):** Vítor Ribeirinho - responsável pela liderança executiva da KPMG Portugal;
- **Head of Function Advisory:** João Sousa - supervisiona as funções de consultoria estratégica;
- **Head of LoB - Tech Consulting:** Rui Gonçalves - lidera a linha de negócio de consultoria tecnológica;
- **Senior Manager (TC Data and Analytics):** Nuno Miranda - gere a equipa de Data & Analytics.

2.2 Equipa de Data & Analytics

Como referido no subcapítulo 1.1, o estágio decorreu na equipa de Data & Analytics da KPMG Portugal. Esta equipa constitui o contexto direto em que o projeto foi desenvolvido, articulando competências em análise de dados, inteligência artificial e soluções tecnológicas aplicadas a necessidades internas e de clientes.

Durante o estágio, o contacto com esta equipa permitiu compreender melhor o modo como ferramentas de AutoML podem ser avaliadas não apenas do ponto de vista técnico, mas também em função da sua aplicabilidade em contextos reais de consultoria.

2.3 Orientação e Acompanhamento

O acompanhamento do estágio foi estruturado de forma a garantir um desenvolvimento progressivo e orientado. A equipa de orientação incluiu:

-
- **Orientador Principal:** Nuno Miranda (Senior Manager) - responsável pela supervisão geral do estágio e alinhamento estratégico do trabalho desenvolvido;
 - **Co-orientador:** Miguel Silvério (Data Scientist) - providenciou orientação técnica diária e acompanhamento próximo do desenvolvimento do projeto. O facto de ser também aluno da Universidade de Évora facilitou a comunicação e compreensão mútua dos objetivos académicos;
 - **Suporte técnico:** Miguel Santos (Data Scientist) - ofereceu apoio especializado em diferentes aspetos técnicos do projeto.

2.4 Metodologia de Trabalho e Colaboração

O trabalho foi estruturado através de reuniões diárias com o co-orientador Miguel Silvério e, frequentemente, com o Nuno Miranda. Estas reuniões permitiam avaliar o progresso, discutir obstáculos encontrados e definir os próximos passos. O processo de *onboarding* incluiu a realização de formações obrigatórias em conformidade, gestão de dados e segurança da informação, seguidas de formações técnicas:

- Introduction to Generative AI (Google Skills, 30 min)
(https://www.skills.google/paths/118/course_templates/536);
- ChatGPT Prompt Engineering for Developers (DeepLearning.AI, 1h30)
(https://www.deeplearning.ai/short-courses/chatgpt-prompt-engineering-for-developers/?utm_source=chatgpt.com);
- LangChain for LLM Application Development (DeepLearning.AI, 1h38)
(<https://www.deeplearning.ai/short-courses/langchain-for-llm-application-development>).

O regime de trabalho foi presencial, com horário completo, e sessões semanais informais de acompanhamento com o orientador principal.

2.5 Inserção no Projeto e Contribuição para Projetos Paralelos

Ao longo do estágio, o trabalho desenvolvido no âmbito do AutoML poderá ser diretamente aproveitado em projetos paralelos da equipa, nomeadamente na avaliação e seleção de ferramentas de análise preditiva para clientes. Esta transversalidade evidencia a aplicabilidade prática do projeto e reforça competências de comunicação técnica e de trabalho em equipa multidisciplinar.

3 Fundamentação Teórica

Este capítulo apresenta a fundamentação teórica que suporta o trabalho desenvolvido. São abordados o conceito e o funcionamento do *Automated Machine Learning* (AutoML), as suas principais aplicações e o equilíbrio entre vantagens e limitações, culminando num levantamento do estado da arte que enquadra as ferramentas e plataformas avaliadas nos capítulos seguintes.

3.1 Conceito de AutoML

O *Automated Machine Learning* (AutoML) é o processo que automatiza as tarefas interativas e complexas da criação de modelos de Aprendizagem Automática. O objetivo principal é reduzir a necessidade de intervenção humana especializada e permitir que utilizadores com diferentes níveis de conhecimento técnico possam desenvolver modelos eficazes e prontos para produção [1,2]. De forma geral, o AutoML inclui mecanismos automáticos para [3]:

- Preparação e pré-processamento de dados;
- Engenharia e seleção de atributos;
- Seleção e comparação de algoritmos;
- Otimização de hiperparâmetros (HPO);
- Criação de comités (*Ensemble Learning*);
- Exportação e deployment de modelos;
- Monitorização e manutenção após a implementação (MLOps Automatizado).

3.2 Funcionamento do AutoML

O AutoML automatiza o ciclo de Aprendizagem Automática em cinco etapas principais:

- **Preparação de dados:** executa automaticamente o tratamento de valores em falta, remoção de valores atípicos, normalização, codificação de variáveis categóricas e divisão dos dados em treino/validação/teste;
- **Engenharia de atributos:** criação de novos atributos derivados, seleção dos mais relevantes e remoção de redundâncias;

-
- **Seleção e treino de modelos:** comparação automática de múltiplos algoritmos (árvores de decisão, *Random Forest*, *Gradient Boosting*, redes neurais, entre outros) com base numa métrica de avaliação definida;
 - **Otimização de hiperparâmetros:** exploração sistemática do espaço de configurações de cada modelo, por métodos como *Bayesian Optimization* ou algoritmos genéticos;
 - **Ensemble modeling e deployment:** combina os melhores modelos em comités (*voting*, *stacking*) e exportação para APIs ou formatos portáteis, com monitorização contínua em produção.

3.3 Aplicações do AutoML

O AutoML aplica-se a uma grande variedade de tarefas: classificação, regressão, agrupamento, previsão de séries temporais, visão computacional e Processamento de Linguagem Natural, cobrindo domínios tão diversos como a deteção de fraude, o diagnóstico médico, a previsão de vendas ou a análise de sentimento.

3.4 Vantagens e Desvantagens do AutoML

Entre as principais vantagens do AutoML destacam-se a redução do tempo de desenvolvimento de modelos, a possibilidade de obter resultados competitivos sem especialistas, o aumento da consistência e a escalabilidade para grandes volumes de dados.

A exploração prática deste estágio confirmou que as plataformas avaliadas produzem modelos com bom desempenho preditivo, embora a qualidade dependa fundamentalmente dos algoritmos subjacentes. Como desvantagens, o AutoML apresenta menor capacidade de personalização face a modelos construídos manualmente, opacidade em alguns modelos criados, custos elevados em plataformas comerciais e o risco de sobreajuste se a experimentação não for devidamente controlada [2,3].

3.5 Estado da Arte

O AutoML emergiu no início dos anos 2000, com trabalhos pioneiros como o Auto-WEKA (2013) que demonstraram ser possível automatizar a seleção de algoritmos sem intervenção humana [1,2]. A partir de 2015, o TPOT introduziu a exportação do *pipeline* como código Python reproduzível e o Auto-sklearn trouxe o conceito de warm starting por meta-aprendizagem, acelerando significativamente a pesquisa [3,4].

Em 2017, o lançamento do Neural Architecture Search (NAS) pela Google estendeu a automação ao design de redes neurais, abrindo o AutoML a problemas de imagem, texto e séries temporais [1,3]. Entre 2019 e 2022, plataformas cloud como o Azure AutoML, o AWS SageMaker Autopilot e o Google Vertex AI democratizaram o acesso, introduzindo interfaces visuais sem código e integração nativa com ecossistemas de dados empresariais.

Atualmente, o AutoML cresce a um ritmo superior a 87% ao ano em publicações acadêmicas [2]. Os principais desafios em aberto são a interpretabilidade dos modelos criados e a robustez fora do domínio de treino. A emergência de modelos de linguagem com capacidade de geração de código representa uma nova fronteira, com potencial para automatizar a criação de *pipelines* inteiramente novos a partir de descrições em linguagem natural.

4 Contexto Operacional

Esta secção descreve o ambiente de hardware e software utilizado durante o desenvolvimento do trabalho de exploração e teste de técnicas de AutoML, incluindo as ferramentas, plataformas e infraestrutura que suportaram o trabalho realizado.

4.1 Ambiente tecnológico

4.1.1 Hardware

O computador atribuído para o desenvolvimento do projeto era um portátil Dell equipado com um processador Intel Core i7-1165G7 de 11ª geração a 2.8GHz, 16GB de memória RAM DDR4 a 4276MHz e placa gráfica integrada Intel Iris Xe Graphics. Este equipamento estava conectado à rede interna da empresa através de uma ligação por cabo Type-C com identificação de endereço MAC para controlo de acesso. A infraestrutura de rede da empresa estava dividida em duas sub-redes no router (uma para a rede por fios interna e outra para a rede sem fios) para evitar o acesso sem fios a serviços internos sem autenticação adequada.

O acesso remoto à rede interna via VPN (Cisco AnyConnect Secure Mobility Client) foi determinante para garantir a continuidade do trabalho fora das instalações da KPMG. A ausência de GPU dedicada e as restrições do sistema operativo Windows condicionaram algumas decisões técnicas, nomeadamente a inviabilidade de utilizar algumas ferramentas open-source como o Auto-sklearn (exclusivo para Linux), o AutoDL (dependente de versões antigas de PyTorch/TensorFlow) e o AutoFolio (também exclusivo para Linux).

4.1.2 Software

O desenvolvimento do trabalho foi realizado com apoio de um conjunto abrangente de bibliotecas especializadas. A gestão do ambiente Python foi assegurada pela distribuição Anaconda, que permitiu criar e isolar ambientes virtuais dedicados a cada ferramenta. O Jupyter Notebook / JupyterLab serviu como ambiente de desenvolvimento integrado principal. As principais bibliotecas e ferramentas Python utilizadas incluíram:

- **Pandas e NumPy:** para manipulação e análise de dados estruturados;
- **Matplotlib e Seaborn:** para criação de visualizações e gráficos comparativos;
- **scikit-learn:** para pré-processamento, métricas de avaliação e modelos de baseline;
- **TPOT:** ferramenta open-source de AutoML baseada em programação genética;

-
- **AutoGluon:** framework de AutoML com suporte a modelos tabulares, visão computacional e NLP;
 - **H2O.ai AutoML:** plataforma open-source para grandes volumes de dados, com suporte a modelos clássicos e aprendizagem profunda;
 - **AdaNet:** framework baseada em TensorFlow para construção iterativa de comités adaptativos.

4.1.3 Ambiente aplicativo

O sistema opera como uma solução autónoma de exploração e avaliação comparativa de ferramentas AutoML, sem integração direta em sistemas de produção da KPMG. O ambiente divide-se em dois componentes:

1. **Módulo de Execução Local:** as ferramentas open-source como TPOT, AutoGluon, H2O AutoML e AdaNet foram executadas em ambientes virtuais Python, com notebooks Jupyter organizados por ferramenta e conjunto de dados, garantindo reprodutibilidade;
2. **Integração Cloud:** foram testadas plataformas como Azure ML AutoML, SageMaker Canvas e Vertex AI AutoML, usando respetivamente Azure Blob Storage, Amazon S3 e Google Cloud Storage/BigQuery, com recursos associados à subscrição da KPMG e a contas de avaliação gratuita.

4.2 Metodologia de Trabalho

Durante a realização do estágio, foi adotada uma metodologia de desenvolvimento iterativa e experimental, adaptada às características específicas do projeto de exploração de ferramentas e às necessidades da equipa.

4.2.1 Processos de Desenvolvimento de Software

O desenvolvimento seguiu uma abordagem híbrida que combinou práticas de investigação experimental com um *modus operandi* ágil. De acordo com o conteúdo e objetivo do projeto, foi implementado um ciclo de desenvolvimento em três fases principais:

1. **Fase de Investigação e Prototipagem:** Cada ferramenta ou plataforma começava com uma fase de investigação das tecnologias disponíveis, levantamento bibliográfico e prototipagem rápida para validação de conceitos;

-
2. **Fase de Implementação Iterativa:** Desenvolvimento incremental dos notebooks de experimentação, com testes contínuos e refinamento baseado nos resultados obtidos;
 3. **Fase de Validação e Documentação:** Análise extensiva dos resultados, comparação entre ferramentas e documentação das descobertas para partilha com a equipa e para integração neste relatório.

4.2.2 Gestão do Processo de Desenvolvimento

A gestão do projeto foi realizada através de reuniões semanais de acompanhamento com o supervisor, onde eram discutidos os progressos, desafios encontrados e próximos objetivos. Entre reuniões, manteve-se comunicação regular através do Microsoft Teams para questões que requeriam discussão imediata ou partilha de ficheiros e notebooks.

A presença física no escritório facilitou significativamente a comunicação, permitindo esclarecimentos rápidos e discussões técnicas espontâneas sempre que surgiam dúvidas ou desafios durante o desenvolvimento. Esta proximidade física revelou-se particularmente valiosa para a resolução de problemas complexos que beneficiavam de explicação visual ou demonstração direta no ecrã.

4.2.3 Planificação do Trabalho

O trabalho foi planificado de forma comunicativa e adaptativa, com objetivos específicos definidos semanalmente em colaboração com o supervisor. A cronologia das atividades realizadas foi a seguinte:

- **Semana 1 (05-06/03):** Realização das formações obrigatórias e técnicas introdutórias. Configuração do ambiente de desenvolvimento e introdução ao projeto de AutoML;
- **Semana 2 (12-13/03):** Levantamento formal do conceito de AutoML, pesquisa de artigos científicos e levantamento das principais ferramentas e frameworks disponíveis;
- **Semana 3 (19-20/03):** Instalação e primeiro teste das ferramentas open-source. Resolução de problemas de compatibilidade (Windows). Inicialmente, apenas foi possível instalar o TPOT;
- **Semana 4 (26-27/03):** Re-teste das ferramentas open-source após resolução dos problemas de ambiente. Instalação e teste do H2O.ai AutoML, AutoGluon e AdaNet. Casos práticos com o conjunto de dados “Iris”;

-
- **Semana 5 (2-3/04):** Análise comparativa das ferramentas open-source com o conjunto de dados "Iris". Elaboração das conclusões da comparação;
 - **Semana 6 (9-10/04):** Estudo dos mecanismos de configuração de cada ferramenta. Validação da comparação com o conjunto de dados "Breast Cancer Wisconsin";
 - **Semana 7 (16-17/04):** Implementação de configuração para as ferramentas. Utilização do F-Score como métrica de avaliação. Análise SWOT comparativa;
 - **Semana 8 (23-24/04):** Início da exploração das plataformas cloud: levantamento de funcionalidades, configuração de contas e compreensão dos fluxos de cada plataforma,
 - **Semana 9 (30/04):** Exploração prática das plataformas cloud, com os mesmos conjuntos de dados utilizados nas ferramentas open-source;
 - **Semana 10 (7-8/05):** Análise comparativa das plataformas cloud, incluindo elaboração da análise SWOT individual de cada plataforma e documentação das implicações para a prática de consultoria;
 - **Semana 11(14-15/05):** Preparação da apresentação e do relatório final sobre todo o projeto desde a concepção até aos resultados e conclusões obtidas.

4.2.4 Interação com os Restantes Membros da Equipa

Como já referido, o trabalho foi desenvolvido em estreita colaboração com o supervisor de estágio, que forneceu orientação técnica e estratégica ao longo de todo o processo. As reuniões semanais serviam não apenas para apresentação do progresso, mas também para discussão de abordagens técnicas e validação de decisões de arquitetura.

A comunicação com outros membros da equipa de investigação da empresa ocorria de forma informal, através de discussões e partilha de conhecimentos. Para desafios técnicos mais específicos, recorreu-se à documentação oficial das plataformas e a fóruns especializados, esta combinação de orientação interna e consulta externa revelou-se adequada para um projeto de natureza exploratória.

5 Trabalho desenvolvido

O estágio teve como objetivo principal explorar e testar técnicas de AutoML, desenvolvendo uma metodologia estruturada de avaliação comparativa entre ferramentas open-source e plataformas cloud. Foram aplicados conjuntos de dados com características propositadamente distintas, mas a utilização de um número limitado de conjuntos de dados ficou condicionada pelas restrições temporais do estágio.

O trabalho iniciou-se com a definição formal dos objetivos e um levantamento bibliográfico sobre o estado da arte em AutoML [1, 2, 3], seguido de uma fase de instalação e configuração que revelou limitações de compatibilidade relevantes. Cada ferramenta de open-source foi aplicada primeiro ao conjunto de dados “Iris”, em condições controladas de baixa complexidade, e depois ao “Breast Cancer Wisconsin”, onde as diferenças entre ferramentas se tornaram mais visíveis. Foi ainda estudada a configuração de cada ferramenta, incluindo o uso de ficheiros externos. Para as plataformas cloud optou-se apenas pelo conjunto de dados “Breast Cancer Wisconsin”, exceto no Vertex AI AutoML que impõe um mínimo de 1000 linhas de treino inviabilizando o uso do “Breast Cancer Wisconsin” (569 amostras), obrigando à introdução de um novo conjunto de dados o “Adult Income” (30 162 registos).

A Tabela 1 apresenta uma caracterização comparativa dos três conjuntos de dados utilizados.

Propriedade	"Iris"	"Breast Cancer Wisconsin"	"Adult Income"
Fonte	UCI / scikit-learn	UCI	UCI
N.º de registos	150	569	30.162
N.º de atributos	4	30	14
Tipo de problema	Classif. multiclasse (3 classes)	Classif. binária (maligno/benigno)	Classif. binária (rendimento >/≤50K)
Tipo de atributos	Todas numéricas contínuas	Todas numéricas contínuas	Mistas (numéricas e categóricas)
Valores em falta	Não	Não	Sim
Desequilíbrio de classes	Equilibrado (50/50/50)	Moderado (63% benigno / 37% maligno)	Moderado (75% ≤50K / 25% >50K)
Complexidade	Baixa	Média	Elevada
Ferramentas utilizadas	TPOT, AutoGluon, H2O, AdaNet, Azure ML, SageMaker Canvas	TPOT, AutoGluon, H2O, AdaNet, Azure ML, SageMaker Canvas	Google Vertex AI AutoML

Tabela 1 – Caracterização comparativa dos conjuntos de dados utilizados no trabalho desenvolvido

5.1 Formação Inicial e Levantamento Teórico

A fase inicial do estágio incluiu a realização das formações obrigatórias da KPMG e de um conjunto de formações técnicas introdutórias relevantes para o projeto referidas anteriormente na subsecção 2.4.

Em paralelo, foi feito um levantamento formal do conceito de AutoML, detalhado no Capítulo 3. Houve uma pesquisa e análise de artigos científicos de referência sobre o estado da arte [1, 2, 3] e estudo da documentação oficial de cada ferramenta. Foi também identificado o domínio de ferramentas AutoML em estudo dividido em soluções open-source e plataformas cloud.

5.2 Exploração Prática das Ferramentas Open-Source

A fase de exploração prática iniciou-se com a instalação e configuração de cada ferramenta. Este processo revelou imediatamente um conjunto de limitações de compatibilidade que condicionaram as escolhas metodológicas subsequentes. Os resultados deste processo de avaliação são apresentados na tabela 2.

Ferramenta	Windows	Estado
TPOT	✓	Instalado
AutoGluon	✓	Instalado
AdaNet	✓	Instalado
Auto-sklearn	X Linux apenas	Não instalado
AutoDL	X Depende de versões antigas do PyTorch e do TensorFlow que não existem para windows	Não instalado
AutoFolio	X Linux apenas	Não instalado

Tabela 2 - Compatibilidade das ferramentas open-source de AutoML com o ambiente Windows 11

Esta análise revelou que o ecossistema open-source de AutoML não é concebido apenas para ambientes Windows, o que restringe a sua adoção ágil em contextos empresariais como o da KPMG, onde este é o sistema operativo padrão. Acresce que, embora a infraestrutura cloud permita contornar estas limitações, os ambientes cloud em contexto empresarial estão sujeitos a restrições rigorosas de cibersegurança que, no âmbito de um estágio curricular, não são viáveis de configurar.

A estratégia de avaliação consistiu na aplicação de cada ferramenta a dois conjuntos de dados de referência: o "Iris", um problema clássico de classificação multiclasse com apenas quatro variáveis contínuas e clara separação entre classes e o "Breast Cancer Wisconsin", um problema de classificação binária com maior número de variáveis.

5.2.1 TPOT

O TPOT (*Tree-based Pipeline Optimization Tool*) [4] em vez de selecionar o melhor algoritmo a partir de um catálogo fixo, o próprio *pipeline* de Aprendizagem Automática é tratado como um organismo sujeito a evolução genética. Cada *pipeline* é avaliado por uma função de aptidão (por defeito, F1-Score em validação cruzada estratificada) e as combinações mais promissoras são selecionadas e alteradas através de operações de recombinação e mutação ao longo de gerações sucessivas, num processo análogo à seleção natural.

5.2.1.1 Configuração

No TPOT, a configuração opera em dois níveis, o espaço de procura e os parâmetros do processo evolutivo. O espaço de procura define quais os algoritmos, transformações e hiperparâmetros que podem integrar os *pipelines* candidatos. Em versões anteriores, era configurável via `config_dict` (um argumento da API Python que aceita um dicionário definido pelo utilizador). Na versão utilizada, este argumento foi descontinuado sem que tenha sido documentada uma alternativa equivalente, o que representa uma limitação significativa. A ferramenta perde flexibilidade de adaptação a domínios específicos, uma característica essencial em contextos empresariais onde o conhecimento de negócio deve poder ser incorporado no processo de modelação.

Os parâmetros do processo evolutivo continuam configuráveis via API. Contudo, a ausência de suporte nativo a ficheiros externos impede a parametrização declarativa e reproduzível que seria esperada de uma ferramenta de nível profissional.

5.2.1.2 Exploração Prática e Análise Crítica - “Iris”

Na aplicação ao “Iris”, o *pipeline* final criado pelo TPOT integrou *StandardScaler*, *VarianceThreshold* e dois blocos *FeatureUnion* com transformações identidade, culminando num classificador Gaussian Naïve Bayes. A convergência para este modelo simples é analiticamente coerente, pois o “Iris” tem forte separabilidade linear nas duas variáveis das pétalas e o GaussianNB representa precisamente essa estrutura.

Contudo, a presença dos blocos *FeatureUnion* com transformações identidade revela uma limitação estrutural do TPOT, o processo evolutivo não distingue eficazmente entre complexidade útil e complexidade resultante de deriva genética (problema designado na literatura como *bloat*).

5.2.1.3 Exploração Prática e Análise Crítica - “Breast Cancer Wisconsin”

No “Breast Cancer Wisconsin”, a maior dimensionalidade (30 atributos derivados de imagens médicas) e a estrutura de classificação binária com desequilíbrio moderado de classes colocaram ao TPOT um problema de natureza diferente. A presença de atributos altamente correlacionados (características de *mean*, *se* e *worst* para cada variável morfológica) cria redundância que afeta algoritmos sensíveis à escala e à multicolinearidade, tornando a seleção automática de atributos e a normalização mais relevantes do que no “Iris”.

O TPOT demonstrou competência na identificação de *pipelines* que abordam esta redundância, mas continuou a evidenciar a tendência para produzir estruturas mais complexas do que o necessário. Em contextos onde os *pipelines* produzidos precisam de ser auditados, mantidos e explicados a stakeholders não técnicos, a opacidade estrutural do TPOT representa um custo operacional que deve ser contabilizado na decisão de adoção.

5.2.1.4 Conclusões

O TPOT é adequado para contextos onde a exportabilidade do modelo como código Python e o baixo custo computacional são prioritários. O seu paradigma evolucionário oferece uma exploração ampla do espaço de soluções, ainda que com tendência para criar estruturas mais complexas do que o necessário.

Vantagens:

- Exploração ampla e não supervisionada do espaço de *pipelines*, sem pressupostos sobre a estrutura ótima;
- *Pipeline* final exportável como código Python legível;
- Custo computacional reduzido;
- Boa cobertura do catálogo clássico de algoritmos scikit-learn, suficiente para a maioria dos problemas tabulares.

Desvantagens:

- Problema de *bloat*: tendência para produzir *pipelines* com etapas redundantes resultantes de deriva genética;
- Descontinuação do `config_dict` na versão atual sem alternativa documentada;

-
- Ausência de comités avançados;
 - Sem suporte nativo a ficheiros externos de configuração.

5.2.2 AutoGluon

AutoGluon [5], desenvolvido pela Amazon Web Services, em vez de procurar o *pipeline* ótimo por procura evolutiva, treina sistematicamente um conjunto diversificado de famílias de modelos (LightGBM, CatBoost, XGBoost, *Random Forest*, *Extra Trees* e redes neurais, entre outros) e combina-os automaticamente em comités de *stacking* multi-nível, partindo do princípio de que a diversidade de modelos reduz, por si só, o viés de seleção.

Nesta ferramenta o utilizador não precisa de especificar qualquer etapa do *pipeline*, pois o AutoGluon trata automaticamente da imputação de valores em falta, codificação de variáveis categóricas, normalização e construção de comités, o que reduz o tempo de entrada em produção.

5.2.2.1 Configuração

A configuração do AutoGluon opera através do parâmetro *hyperparameters* da função `fit()`, que aceita um dicionário especificando, quais as famílias de modelos a incluir e os intervalos de hiperparâmetros a explorar. O parâmetro *presets* (com opções como *best_quality*, *high_quality*, *medium_quality* e *fast*) controla o equilíbrio entre profundidade de otimização e tempo de execução, tornando a ferramenta adaptável a diferentes contextos operacionais.

A externalização desta configuração para ficheiro JSON é tecnicamente possível, mas não nativa, o utilizador lê o ficheiro em Python e passa o conteúdo como argumento sem validação prévia da configuração, o que em *pipelines* de MLOps aumenta o risco de falhas silenciosas.

5.2.2.2 Exploração Prática e Análise Crítica - “Iris”

No “Iris”, o AutoGluon treinou todos os modelos do seu catálogo e construiu um *WeightedEnsemble_L2* como o melhor modelo, com o *NeuralNetFastAI* a receber o peso dominante. Num problema de baixa complexidade como o “Iris”, o ganho do comité face ao melhor modelo individual tende a ser marginal, qualquer algoritmo razoável atinge desempenho elevado, e a separabilidade linear dos dados não deixa muito espaço para que a diversidade do comité se traduza em vantagens reais.

Este resultado indica que a qualidade de um comité depende da diversidade das suas componentes e não da sua quantidade. O que acaba por ser relevante, pois não podemos associar automaticamente "mais modelos" a "melhor resultado".

5.2.2.3 Exploração Prática e Análise Crítica - “Breast Cancer Wisconsin”

No “Breast Cancer Wisconsin”, o comportamento do AutoGluon mudou qualitativamente. A maior diversidade estrutural do conjunto de dados criou condições para que diferentes algoritmos identificassem diferentes aspetos dos dados. Os modelos baseados em *gradient boosting* (LightGBM, XGBoost) tendem a ser mais eficazes em não linearidades numéricas, enquanto as redes neurais identificam interações de ordem superior. O modelo final foi também o *WeightedEnsemble_L2* juntamente com o LightGBMXT.

Este contraste com os resultados no “Iris”, demonstra que o valor de uma ferramenta de AutoML não pode ser avaliado com base num único conjunto de dados.

5.2.2.4 Conclusões

O AutoGluon é a escolha preferencial quando o desempenho é o critério dominante e os dados apresentam complexidade média a elevada. O seu elevado nível de automatização reduz o tempo de entrada, ainda que à custa de maior opacidade e custo computacional.

Vantagens:

- Automatização completa do *pipeline*;
- Tabela de classificação detalhada com análise de importância de atributos;
- Comité de *stacking* multi-nível com benefícios reais em problemas de complexidade média a alta;
- *Presets* configuráveis.

Desvantagens:

- Custo computacional elevado;
- Opacidade do comité dificulta auditoria;
- Risco de sobreajuste em conjunto de dados pequenos;
- Configuração via ficheiro JSON sem validação nativa.

5.2.3 H2O AutoML

O H2O AutoML [6] assume que o valor de um modelo não reside apenas na sua performance, mas também na capacidade de explicar e defender as suas previsões, uma perspectiva particularmente relevante em consultoria, onde a explicabilidade perante clientes e auditores é muitas vezes um requisito.

Internamente, treina um conjunto diversificado de algoritmos (GBM, XGBoost, *Random Forest*, *Deep Learning* e GLM) e constrói dois comités alternativos: o *StackedEnsemble_AllModels*, que agrega todos os modelos treinados, e o *StackedEnsemble_BestOfFamily*, que seleciona o melhor representante de cada família. O H2O compara ambos e propõe o melhor como modelo final.

5.2.3.1 Configuração

A configuração do H2O AutoML é feita através dos parâmetros do objeto `H2OAutoML`, que incluem `max_runtime_secs`, `max_models`, `seed`, `sort_metric` e `exclude_algos`. Este último é particularmente útil em contextos empresariais, pois permite por exemplo, excluir modelos de *Deep Learning* quando a interpretabilidade é um requisito, ou excluir GLM quando o problema apresenta padrões claramente não lineares.

A externalização para ficheiro JSON requer código Python intermediário, e a necessidade de inicializar um servidor H2O local (JVM) pode ser problemática em ambientes de execução automatizada sem permissões de administrador.

5.2.3.2 Exploração Prática e Análise Crítica - “Iris”

No “Iris”, identificou como melhor modelo um *StackedEnsemble_AllModels*, evidenciando a sua preferência por soluções robustas mesmo em problemas simples. A curva de aprendizagem apresentou comportamento estável sem sinais de sobreajuste, e os gráficos de dependência parcial (PDP) confirmaram a dominância das variáveis das pétalas na separação das classes. Os PDPs assumem independência entre atributos ao calcular o efeito marginal de cada variável, o que pode induzir interpretações incorretas em dados com correlações fortes.

5.2.3.3 Exploração Prática e Análise Crítica - “Breast Cancer Wisconsin”

No “Breast Cancer Wisconsin”, o H2O AutoML confirmou o padrão observado no AutoGluon, a maior complexidade do conjunto de dados permite tirar melhor partido das estratégias de comité. A disponibilidade de PDPs, curvas de aprendizagem e tabela de

modelos candidatos ordenada por F-Score tornou o H2O a ferramenta com maior riqueza analítica entre as quatro avaliadas, num projeto de consultoria, a capacidade de apresentar não apenas o modelo mas também a sua justificação técnica detalhada representa um diferencial de valor significativo.

A comparação dos dois comités produzidos (*AllModels* e *BestOfFamily*) revelou diferenças subtis de desempenho que variam consoante o conjunto de dados, sugerindo que a estratégia de seleção de modelos para *stacking* merece atenção em aplicações reais.

5.2.3.4 Conclusões

O H2O AutoML representa um bom equilíbrio entre desempenho e interpretabilidade. A riqueza dos seus resultados analíticos torna-o particularmente adequado para projetos de consultoria onde a explicação dos modelos perante clientes é um requisito.

Vantagens:

- Outputs analíticos ricos;
- Parâmetro `exclude_algos` permite incorporar conhecimento de domínio;
- Orientação para produção com estabilidade e reprodutibilidade.

Desvantagens:

- Inicialização de servidor H2O local (JVM);
- Configuração externa via JSON não nativa;
- Os PDPs assumem independência entre atributos;
- A complexidade dos comités dificulta a auditoria em contextos regulados.

5.2.4 AdaNet

O AdaNet [7] é a ferramenta que mais se afasta da noção convencional de AutoML. Enquanto as restantes automatizam a seleção e comparação de algoritmos heterogéneos, o AdaNet foca-se exclusivamente na construção adaptativa de comités de redes neuronais, exigindo que o utilizador defina previamente o conjunto de arquiteturas candidatas (`candidate_pool`). Isto coloca-o numa categoria distinta e não num sistema de AutoML de propósito geral.

O algoritmo constrói o comitê iterativamente, adicionando em cada ronda a sub-rede que maximiza um limite superior garantido de generalização.

5.2.4.1 Configuração

A configuração do AdaNet centra-se na definição do `candidate_pool`. Os parâmetros principais incluem `hidden_units`, `train_steps` e `max_iteration_steps`. A dependência de versões específicas do TensorFlow e da API *Estimator* (descontinuada nas versões recentes) constitui uma limitação estrutural severa, pois qualquer atualização do ambiente pode perder a compatibilidade, tornando o AdaNet difícil de manter em produção.

A utilização de ficheiro JSON externo é possível mas requer tradução manual para objetos TensorFlow, eliminando as vantagens de auditabilidade e reprodutibilidade que se associam normalmente à configuração declarativa.

5.2.4.2 Exploração Prática e Análise Crítica - “Iris”

No “Iris”, o AdaNet produziu resultados comparáveis às restantes ferramentas, com apenas um erro de classificação na fronteira versicolor/virginica (o mesmo padrão observado nas outras abordagens). O que indica que em problemas de baixa complexidade, a sofisticação arquitetural adicional do AdaNet não se traduz em qualquer vantagem preditiva.

A configuração e interpretação dos resultados exigiram um esforço significativamente superior ao das restantes ferramentas, uma vez que os resultados produzidos são orientados para o diagnóstico interno do TensorFlow e não para a análise de desempenho do modelo.

5.2.4.3 Exploração Prática e Análise Crítica - “Breast Cancer Wisconsin”

No “Breast Cancer Wisconsin”, o AdaNet manteve o padrão identificado no “Iris”, um desempenho funcional sem vantagens claras face a abordagens mais simples, acompanhado de um custo de configuração desproporcionado. A dependência de versões específicas do TensorFlow e da API *Estimator* tornou o processo de configuração iterativo e frágil, com erros de compatibilidade que exigiram ajustes manuais ao ambiente.

Do ponto de vista da aplicabilidade empresarial, o AdaNet apresenta uma elevada exigência técnica, fraca interpretabilidade dos resultados e dependência de tecnologias em processo de descontinuação, sem ganhos de desempenho demonstráveis nos conjuntos de dados testados. O seu valor reside essencialmente em contextos de investigação onde a construção adaptativa de comités neuronais é o objeto de estudo em si mesmo.

5.2.4.4 Conclusões

O AdaNet é uma ferramenta de investigação especializada, não uma solução de AutoML de propósito geral. O seu valor reside em contextos académicos onde a construção adaptativa de comités neuronais com garantias teóricas é o objeto de estudo, a sua adoção em projetos de consultoria é de difícil justificação face às alternativas disponíveis.

Vantagens:

- Fundamentação teórica robusta com garantias de generalização;
- Flexibilidade na definição de arquiteturas candidatas;
- Potencial em problemas de grande escala onde a compacidade do comité é um requisito.

Desvantagens:

- Não é AutoML de propósito geral;
- Forte dependência da API Estimator do TensorFlow, em processo de descontinuação;
- Outputs orientados para diagnóstico interno e não para análise de desempenho;
- Relação custo/benefício desfavorável em problemas de complexidade moderada.

5.3 Análise Comparativa das Ferramentas Open-Source

A análise comparativa das quatro ferramentas open-source constitui o núcleo metodológico desta secção. A aplicação de todas as ferramentas ao mesmo conjunto de dados, em condições controladas e com os mesmos critérios de avaliação permite isolar o efeito da ferramenta da variabilidade introduzida pelos dados, produzindo conclusões mais robustas do que as que seriam possíveis com um único conjunto de dados.

5.3.1 “Iris” - Isolamento do Efeito da Ferramenta

O conjunto de dados “Iris” foi escolhido como primeiro ponto de comparação por ser um problema bem compreendido, com forte separabilidade entre classes e ausência de ruído relevante. A estratégia de avaliação foi uniforme, com uma divisão 80/20 com semente fixa e F1-Score ponderado como métrica principal.

Os resultados confirmaram que em problemas simples, todas as ferramentas atingem desempenhos equivalentes. Esta convergência demonstra que, no limite de baixa

complexidade, o AutoML reduz a seleção de modelos a um problema trivial. A questão relevante desloca-se, portanto, para a facilidade de configuração, a qualidade dos resultados analíticos, o custo computacional e a interpretabilidade. Nestas dimensões, as diferenças são notórias, o AutoGluon e o H2O destacam-se pela tabela de modelos candidatos e pela análise de importância de atributos, o TPOT pela exportação de *pipelines* como código Python e o AdaNet pelo maior esforço de configuração e pelos resultados menos legíveis.

5.3.2 “Breast Cancer Wisconsin” - Emergência das Diferenças Reais

O conjunto de dados “Breast Cancer Wisconsin” representa um nível de complexidade intermédio, com desequilíbrio moderado de classes e uma elevada correlação entre variáveis. Neste contexto, as diferenças entre as ferramentas tornaram-se mais pronunciadas, o AutoGluon e o H2O AutoML beneficiaram da maior complexidade estrutural com os seus mecanismos de *stacking*, o TPOT manteve desempenho competitivo, mas com *pipelines* mais complexos que evidenciaram o problema de *bloat*, o AdaNet apresentou desempenho equivalente às restantes, mas com custo de configuração desproporcionado.

É ainda relevante notar que um erro de classificação no “Breast Cancer Wisconsin” (em particular um falso negativo) tem potencial impacto clínico, ao contrário do que acontece no “Iris”. Esta diferença de contexto reforça a importância de selecionar a ferramenta não apenas com base no desempenho global, mas também na sua capacidade de fornecer diagnósticos de confiança e mecanismos de controlo do limiar de decisão.

5.3.3 Resultados Comparativos

A Tabela 3 sintetiza os modelos finais selecionados por cada ferramenta e os respetivos F1-Scores nos dois conjuntos de dados. A nota “BCW” refere-se ao conjunto de dados “Breast Cancer Wisconsin”.

Ferramenta	Modelo Final ("Iris")	F1 "Iris"	Modelo Final ("BCW")	F1 "BCW"	Divisão Treino/Teste
TPOT	<i>GaussianNB</i>	0,97	<i>GaussianNB</i>	0,96	80/20
AutoGluon	<i>NeuralNetFastAI</i> (<i>WeightedEnsemble_L2</i>)	1,00	<i>WeightedEnsemble_L2</i> (<i>LightGBMXT</i>)	0,98	80/20 + bagging 8 folds
H2O AutoML	<i>DeepLearning</i>	0,96	<i>GLM</i>	0,98	80/20 (Divisão interna H2O)
AdaNet	<i>LinearEstimator</i>	0,86	<i>TensorFlow DNN</i>	0,95	80/20

Tabela 3 - Resultados comparativos das ferramentas open-source nos conjuntos de dados "Iris" e "Breast Cancer Wisconsin": modelo final selecionado e F1-Score no conjunto de teste

5.3.4 Síntese Crítica, SWOT Comparativa e Recomendações de Uso

A síntese da análise comparativa organiza-se segundo duas dimensões, a adequação ao problema (função da complexidade dos dados e dos requisitos de desempenho) e a adequação ao contexto operacional (função dos requisitos de interpretabilidade, manutenção e integração). Para sistematizar esta análise, apresenta-se de seguida uma análise SWOT individual para cada ferramenta (tabela 4, 5, 6 e 7).

TPOT

Forças	Fraquezas
- Exploração evolutiva do espaço de <i>pipelines</i> sem pressupostos; - <i>Pipeline</i> final exportável como código Python; - Baixo custo computacional.	- Problema de <i>bloat</i>; - <code>config_dict</code> descontinuado; - Erros de configuração só detetados em execução.
Oportunidades	Ameaças
- Integração com CI/CD via exportação de código; - Evolução do ecossistema pode repor o suporte a <code>config_dict</code> em versões futuras; - Ferramenta de exploração inicial em projetos de consultoria.	- Descontinuação de funcionalidades sem garantia de suporte futuro; - Plataformas cloud podem torná-lo redundante; - Menor competitividade em dados de alta dimensionalidade.

Tabela 4 - Análise SWOT do TPOT

AutoGluon

Forças	Fraquezas
- Automatização completa; - Comité multi-nível com benefícios reais em dados de complexidade média-alta; - Tabela de classificação detalhada; - <i>Presets</i> configuráveis.	- Custo computacional elevado; - Opacidade do stacking; - Comité pode ser funcionalmente equivalente a um único modelo em dados simples; - Configuração JSON sem validação nativa.
Oportunidades	Ameaças
- Crescente complexidade dos dados empresariais favorece comités; - Adequado quando desempenho preditivo é critério dominante; - Integração com MLOps.	- Risco de sobreajuste em dados pequenos; - Custo computacional pode ser proibitivo com orçamento restrito; - Complexidade dificulta a auditoria em contextos regulados.

Tabela 5 - Análise SWOT do AutoGluon

H2O AutoML

Forças	Fraquezas
- Outputs analíticos ricos; - Dois tipos de comitê com estratégias diferenciadas; - <code>exclude_algos</code> permite incorporar conhecimento de domínio.	- Inicialização do H2O local como ponto de falha; - Configuração JSON não nativa; - PDPs assumem independência entre atributos.
Oportunidades	Ameaças
- Interpretabilidade avançada é diferencial em contextos regulados; - Riqueza analítica facilita a defesa técnica perante auditores; - <code>exclude_algos</code> alinha a ferramenta com requisitos de explicabilidade do cliente.	- Plataformas cloud com SHAP integrado podem substituí-lo em produção; - PDPs podem induzir decisões incorretas se não for comunicada ao cliente; - Custo de manutenção da integração JVM em ambientes com atualizações frequentes.

Tabela 6 - Análise SWOT do H2O AutoML

AdaNet

Forças	Fraquezas
- Fundamentação teórica robusta com garantias de generalização; - Construção adaptativa e compacta do comitê; - Flexibilidade na definição de arquiteturas candidatas.	- Não é AutoML de propósito geral; - Dependência da API <i>Estimator</i> do TensorFlow, em processo de descontinuação; - Relação custo/benefício desfavorável.
Oportunidades	Ameaças
- Contextos de investigação sobre a construção adaptativa de comitês neuronais; - Problemas de grande escala onde a compacidade é requisito crítico.	- Incompatibilidades de versões do TensorFlow; - Ferramentas mais simples produzem resultados equivalentes com menor esforço.

Tabela 7 - Análise SWOT do AdaNet

A análise SWOT revela que cada ferramenta ocupa um nicho distinto. O TPOT é mais adequado quando a exportação do *pipeline* como código e o baixo custo computacional são prioritários. O AutoGluon é a escolha natural quando o desempenho preditivo é o critério dominante e os dados apresentam complexidade média-alta. O H2O AutoML representa o melhor equilíbrio entre desempenho e interpretabilidade, sendo preferencial quando a explicabilidade perante clientes é um requisito. O AdaNet não revelou vantagens práticas nos conjuntos avaliados, sendo mais adequado a contextos de investigação.

5.4 Exploração Prática das Plataformas Cloud

Para além das ferramentas open-source, foram exploradas três plataformas cloud de AutoML (Azure ML AutoML, AWS SageMaker Canvas e Google Vertex AI AutoML). Estas plataformas foram analisadas apenas com o conjunto de dados “Breast Cancer Wisconsin” por ser o mais complexo. Ao contrário das soluções open-source, estas plataformas oferecem interfaces gráficas no-code.

5.4.1 Azure ML AutoML

O Azure Machine Learning (Azure ML) [8] é a plataforma de Aprendizagem Automática da Microsoft, integrada no ecossistema Azure e acessível através do Microsoft Foundry. Disponibiliza ferramentas desde a preparação de dados até à produção, com suporte para utilizadores técnicos e para utilizadores sem perfil de programação (via interface gráfica AutoML).

O módulo de AutoML permite treinar modelos de forma automatizada, explorando múltiplos algoritmos e combinações de hiperparâmetros sem intervenção manual. Em contrapartida à configuração explícita de infraestrutura que exige, oferece maior transparência, controlo e acesso ao código produzido internamente.

5.4.1.1 Conjunto de Dados e Importação

O conjunto de dados “Breast Cancer Wisconsin” foi importado para a plataforma através do módulo de dados do Azure ML Studio, num processo elaborado de sete passos. O ficheiro foi registado como um asset tabular com o nome “Breast Cancer Wisconsin” e carregado diretamente a partir do disco local, embora a plataforma suporte também importação a partir de Azure Storage, bases de dados SQL e fontes web.

A plataforma detetou automaticamente o formato do ficheiro, um CSV delimitado por vírgula, encoding UTF-8, com headers na primeira linha. Também inferiu os tipos de cada coluna, `id` como inteiro, `diagnosis` como texto e os 30 atributos numéricos como decimais. A coluna `Path`, produzida internamente para rastreio do ficheiro, foi excluída do esquema de treino.

Após confirmação, o conjunto de dados ficou registado como `BCDS:1`, assim o ficheiro fica armazenado no Azure Blob Storage do espaço de trabalho e acessível para qualquer tarefa futura sem necessidade de um novo carregamento.

5.4.1.2 Infraestrutura e Computação

O Azure ML exige a configuração explícita de um recurso de computação antes de submeter qualquer tarefa de treino. A plataforma disponibiliza dois tipos, o *compute instances*, para desenvolvimento e experimentação em notebooks e o *compute clusters* que varia automaticamente entre 0 e N nós, conforme a carga de trabalho. Quando inativos, escalam para 0 nós e não produzem custos.

Para este projeto foi criado um *compute cluster* com a VM `Standard_DS3_v2` (4 núcleos, 14 GB de RAM, \$0.27/hora), adequada para treino de modelos clássicos em conjuntos de dados de pequena dimensão como o “Breast Cancer Wisconsin”. A tabela 8 apresenta as principais opções disponíveis na plataforma, incluindo alternativas de maior capacidade para conjuntos de dados de maior volume e opções GPU para modelos de *Deep Learning*.

VM Size	Categoria	Workload / Custo
Standard_DS11_v2 (2 cores)	Memory optimized	Notebooks e testes ligeiros \$0.19/hr
Standard_DS3_v2 (4 cores, 14GB) [USADO]	General purpose	ML clássico em conjuntos de dados pequenos \$0.27/hr
Standard_E4ds_v4 (4 cores, 32GB)	Memory optimized	Conjuntos de dados médios 1-10 GB \$0.35/hr
Standard_D13_v2 (8 cores, 56GB)	Memory optimized	Conjuntos de dados grandes >10 GB \$0.76/hr

Tabela 8 – Opções de compute cluster, custos da mesmas e desempenho

5.4.1.3 Resultados do Modelo

Concluída a execução, o separador Overview apresenta um resumo completo da tarefa (Figura 1). É também nesta fase que o utilizador acede ao modelo registado, ao conjunto de dados de treino produzido internamente e ao painel lateral com todas as métricas de avaliação calculadas automaticamente sobre o conjunto de validação (Figura 2).

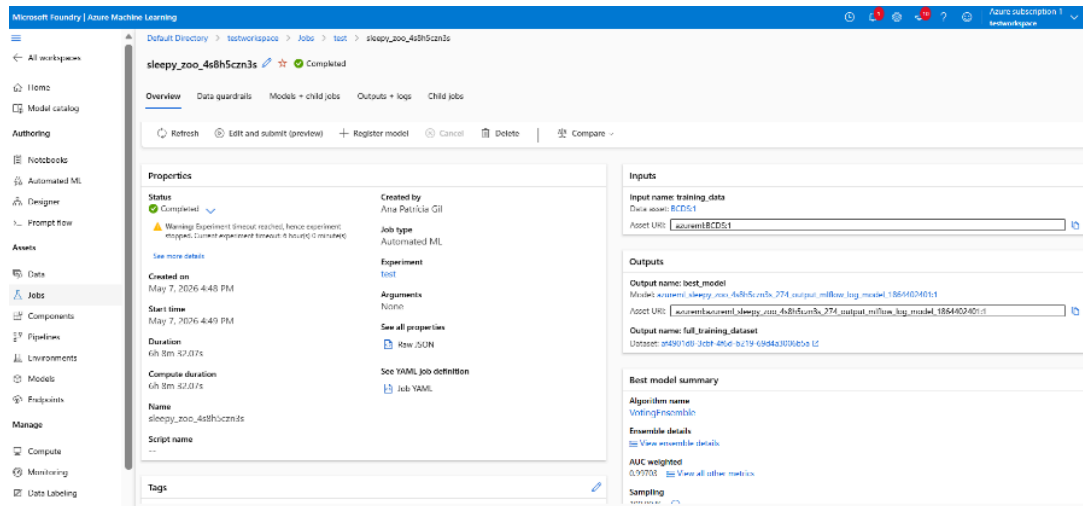


Figura 1 - Overview do modelo

The screenshot shows a vertical list of performance metrics for a model. The metrics and their values are as follows:

Accuracy	0.98050
AUC micro	0.99703
AUC macro	0.99710
AUC weighted	0.99703
Average precision score macro	0.99731
Average precision score micro	0.99720
Average precision score weighted	0.99737
Balanced accuracy	0.97650
F1 score macro	0.97866
F1 score micro	0.98050
F1 score weighted	0.98044
Log loss	0.066657
Matthews correlation	0.95838
Norm macro recall	0.97300
Precision score macro	0.98198
Precision score micro	0.98058

Figura 2 - Painel lateral de métricas

O Azure ML AutoML calcula um conjunto muito alargado de métricas de avaliação abrangendo diferentes perspetivas de qualidade do modelo, como métricas de discriminação, de classificação, de ajustamento probabilístico e de correlação. Esta riqueza de informação permite ao utilizador escolher a perspetiva de avaliação mais adequada ao problema em causa. Em contexto médico, por exemplo, existem métricas mais relevantes do que a simples exatidão global, e o Azure ML AutoML disponibiliza todas de forma imediata, sem necessidade de configuração adicional.

5.4.1.3.1 Modelo Vencedor e Ensemble Details

O melhor modelo selecionado pelo Azure AutoML foi um `VotingEnsemble`, construído depois de treinar e avaliar os modelos candidatos individualmente, a plataforma combina os mais promissores num modelo composto, tendencialmente mais robusto e generalizável do que qualquer um dos seus constituintes isolados.

Neste caso, o algoritmo dominante nos modelos constituintes foram as Máquinas de Vetores de Suporte (SVM), particularmente eficazes em conjuntos de dados de dimensão moderada com atributos numéricos bem estruturados. A diversidade do comité é assegurada pela variação do método de pré-processamento aplicado antes da SVM. `StandardScalerWrapper`, `RobustScaler` e `MaxAbsScaler` normalizam os dados de formas distintas, introduzindo variações que se complementam na decisão final, com cada componente a contribuir com o mesmo peso para a votação.

O painel Ensemble details (Figura 3) permite inspecionar individualmente cada componente. O painel Configuration settings regista todos os parâmetros que regularam a tarefa.



Figura 3 - Ensemble details: lista de componentes do VotingEnsemble

5.4.1.4 Tabela de Classificação

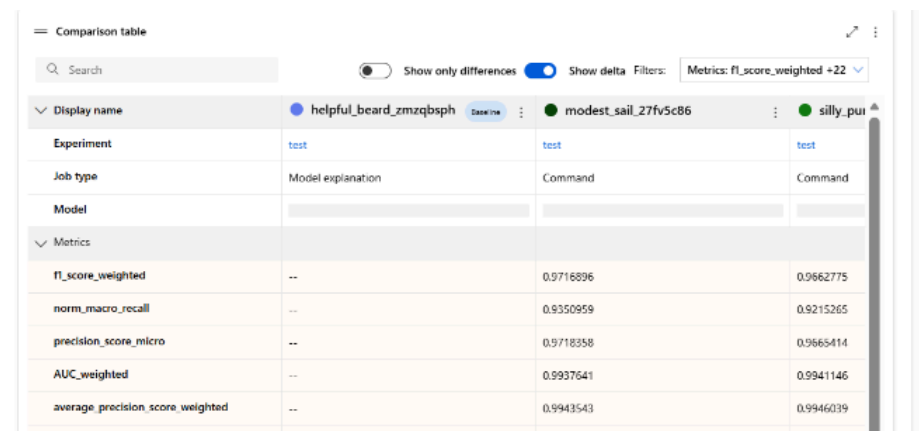
O separador Models + child jobs apresenta a tabela de classificação completa com todos os modelos candidatos explorados durante o job de AutoML, ordenados por AUC weighted (métrica de avaliação pré definida pela plataforma para problemas de classificação binária) de forma decrescente. Esta tabela de classificação é uma das funcionalidades mais relevantes do Azure ML, estando sempre acessível independentemente das configurações escolhidas.

Ao longo das 6 horas de execução, a plataforma explorou múltiplos modelos com o `VotingEnsemble` a ocupar o primeiro lugar com um AUC ponderado de 0.99703. Cada modelo pode ser selecionado para colocação em produção, download ou inspeção detalhada dos hiperparâmetros, sem necessidade de re-treino.

5.4.1.5 Outputs, Logs e Experiment View

No Azure ML no separador Outputs + logs, disponibiliza os artefactos produzidos durante a execução. O ficheiro mais relevante é o `automl_driver.py` (script Python criado automaticamente) que conduz todo o processo de AutoML, desde a importação dos módulos do SDK `azureml.train.automl` até à configuração do *pipeline* de treino, disponível na pasta Azure ML AutoML do GitHub pessoal. Estão também disponíveis os ficheiros `definition_original.json` e `definition.json` com a configuração completa da tarefa em formato legível, tornando o processo auditável e reproduzível via SDK.

O Azure ML organiza todos os trabalhos numa experiência, criando automaticamente um painel interativo que agrega e compara os modelos candidatos. A funcionalidade Job comparison (Figura 4) permite ainda comparar múltiplos candidatos lado a lado com deltas relativos a uma linha de base definida pelo utilizador.



The screenshot displays the 'Comparison table' interface in Azure ML. It features a search bar, toggle for 'Show only differences', a 'Show delta' button, and a filter for 'Metrics: f1_score_weighted +22'. The table lists three experiments with their respective metrics.

Display name	helpful_beard_zmqzbsph	modest_sail_27fv5c86	silly_pui
Experiment	test	test	test
Job type	Model explanation	Command	Command
Model			
Metrics			
f1_score_weighted	--	0.9716896	0.9662775
norm_macro_recall	--	0.9350959	0.9215265
precision_score_micro	--	0.9718358	0.9665414
AUC_weighted	--	0.9937641	0.9941146
average_precision_score_weighted	--	0.9943543	0.9946039

Figura 4 - Job comparison table

5.4.1.6 Conclusões

O Azure ML AutoML posiciona-se para projetos onde a auditabilidade, a reprodutibilidade e a integração com MLOps são requisitos. É a plataforma mais adequada para setores regulados, beneficiando da transparência total do processo e da integração nativa com o ecossistema Microsoft.

Vantagens:

- Transparência total: acesso ao código Python criado, JSONs de configuração e hiperparâmetros de cada candidato;
- Tabela de Classificação sempre disponível;
- Painel Ensemble details expõe os componentes e os hiperparâmetros do `VotingEnsemble`;
- Integração nativa com MLOps para *pipelines* de produção empresariais.

Desvantagens:

- Configuração obrigatória de *compute cluster* antes de qualquer treino;
- Tempo de treino longo (mais de 6 horas), penalizando a prototipagem rápida;
- Curva de aprendizagem elevada, devido à introdução de muitos conceitos novos (workspaces, experiments, data assets, compute clusters);
- Dependência do ecossistema Azure com custos variáveis por hora de computação.

5.4.2 AWS SageMaker Canvas

O Amazon SageMaker Canvas [9] é a plataforma de AutoML da Amazon Web Services criada para utilizadores sem perfil técnico que necessitam de criar modelos preditivos sem depender de equipas de ciência de dados.

O módulo de AutoML automatiza todo o processo desde a importação até à avaliação de desempenho e criação de explicabilidade via valores SHAP. O utilizador interage apenas com uma interface gráfica, sem exposição de detalhes de infraestrutura, algoritmos ou hiperparâmetros.

5.4.2.1 Conjunto de Dados e Importação

O conjunto de dados “Breast Cancer Wisconsin” foi importado para a plataforma através do módulo Datasets do SageMaker Canvas, por carregamento direto do ficheiro CSV. Este processo é automatizado, o utilizador seleciona o ficheiro e a plataforma trata do resto sem qualquer configuração adicional. Os tipos de cada coluna são detetados automaticamente, os valores em falta são identificados e assinalados, e são criadas estatísticas descritivas e histogramas de distribuição para cada variável, acessíveis através do Data Wrangler.

5.4.2.2 Configuração do Modelo

Antes do treino, o Canvas disponibiliza um painel de configuração com duas secções, Basic e Advanced. O processo não requer configuração de infraestrutura, e a maioria das opções têm valores padrão recomendados que permitem começar a treinar de imediato.

O tipo de modelo foi automaticamente sugerido como ‘2 category model’, o que é adequado a uma classificação binária. A métrica de otimização predefinida é o F1, robusta em contextos de dados potencialmente desequilibrados. A divisão dos dados seguiu o padrão 80/20 entre treino e validação, com um máximo de 100 candidatos a explorar. Foi utilizado o modo *Quick Build*, que resulta num treino rápido (2 a 15 minutos) mas com uma exploração menos exaustiva do espaço de pesquisa do que o *Standard Build*. A tabela 9 resume os parâmetros utilizados.

Parâmetro	Valor configurado
Model type	2 category model (detetado automaticamente)
Objective metric	F1 (padrão recomendado)
Training method	Auto (recomendado)
Data split	80% treino / 20% validação
Max candidates	100
Build mode	<i>Quick Build</i> (2-15 minutos)
Infraestrutura	Totalmente criada (invisível ao utilizador)

Tabela 9 – Passos para a criação do modelo

5.4.2.3 Resultados do Modelo

Concluído o treino em modo *Quick Build*, o Canvas apresenta os resultados no separador Analyze, organizado em quatro sub-separadores: Overview (importância dos atributos), Scoring (previsões vs. valores reais), Advanced Metrics e Scoring Details (matriz de confusão e curva Precisão-Recall). Esta plataforma passa o utilizador diretamente para a análise do modelo sem expor detalhes do processo de treino.

5.4.2.3.1 Importância dos Atributos (SHAP)

O sub-separador Overview apresenta o Column Impact, que demonstra a importância de cada atributo para as previsões do modelo, calculada através de valores SHAP. O SHAP atribui a cada atributo uma contribuição individual para cada previsão, permitindo perceber quais as variáveis mais relevantes e de que forma influenciam cada classe.

O atributo mais importante identificado foi `area_worst`, que representa a maior área celular observada. O padrão encontrado é biologicamente coerente, pois valores baixos de `area_worst` contribuem para a previsão de Benigno, enquanto valores altos contribuem para Maligno, o que é consistente com o conhecimento clínico, dado que células malignas tendem a ser maiores e mais irregulares.

5.4.2.3.2 Scoring e Matriz de Confusão

O sub-separador Scoring apresenta o diagrama de Sankey (Figura 5, 7 e 8), uma visualização de fluxo que mostra a distribuição entre previsões corretas e incorretas para cada classe.

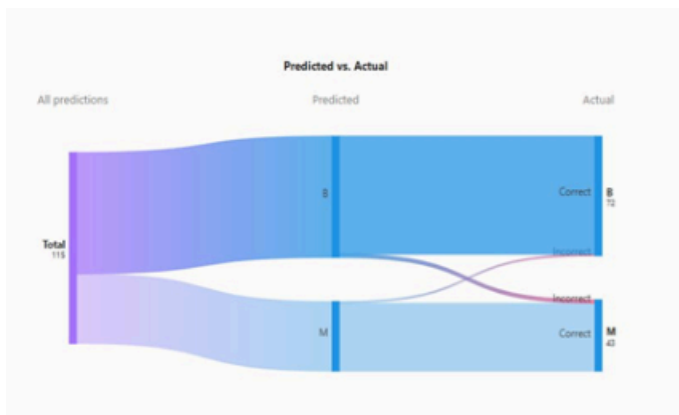


Figura 5 - Diagrama de Sankey

A matriz de confusão (Figura 6), disponível no sub-separador Scoring Details, apresenta os quatro quadrantes clássicos: Verdadeiros Positivos (casos malignos corretamente

identificados), Verdadeiros Negativos (casos benignos corretamente identificados), Falsos Positivos (falsos alertas – Figura 7) e Falsos Negativos (casos malignos não detetados – Figura 8).

Num contexto médico, os Falsos Negativos são o erro mais crítico, pois um caso maligno classificado como benigno pode atrasar o tratamento com consequências graves.

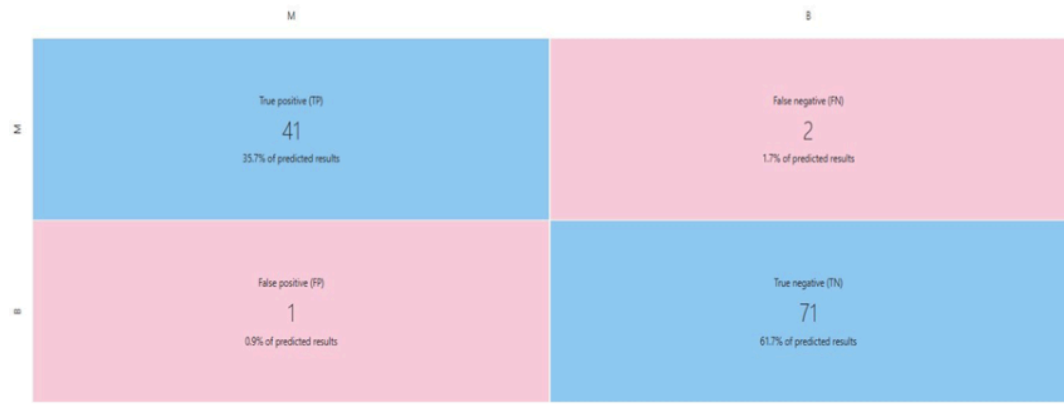


Figura 6 - Matriz de Confusão

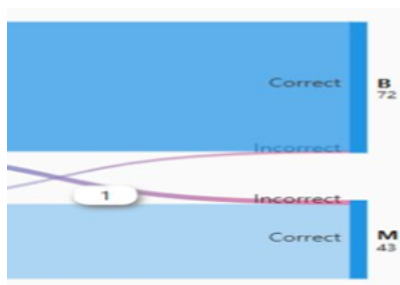


Figura 7 - Diagrama de Sankey: 1 Falso Negativo

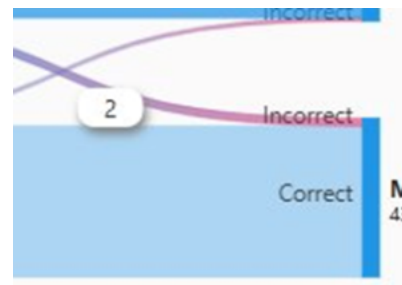


Figura 8 - Diagrama de Sankey: 2 Falsos Positivos

5.4.2.3.3 Métricas Avançadas e Curva Precisão-Recall

O sub-separador Advanced Metrics expõe o conjunto completo de métricas de avaliação do modelo (Figura 9), com a possibilidade de selecionar a classe positiva (B ou M) para adaptar à perspetiva de análise.

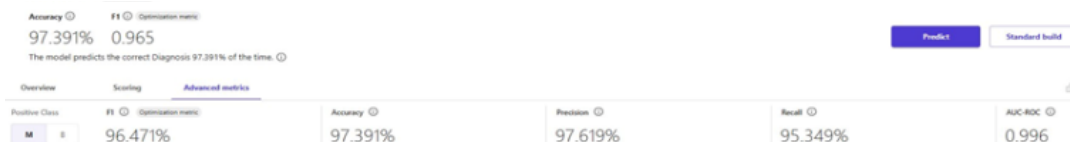


Figura 9 - Métricas Avançadas

A curva Precisão-Recall, também disponível neste separador, complementa as métricas disponíveis com uma perspetiva gráfica do comportamento entre precisão e recall ao longo de diferentes thresholds de classificação (Figura 10). A área sob a curva (AUC-ROC) sintetiza este comportamento num único valor, quanto mais próxima de 1, melhor o modelo em todos os thresholds possíveis.

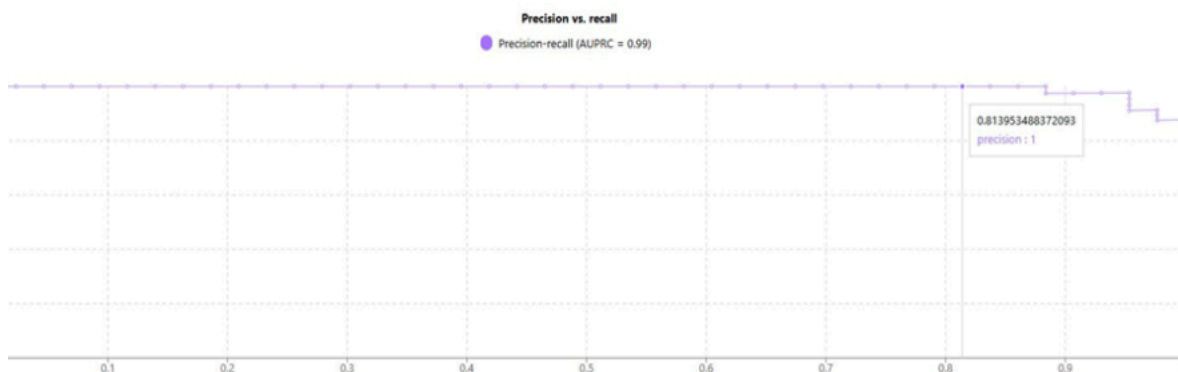


Figura 10 - Curva Precision vs. Recall

5.4.2.4 Tabela de Classificação

O SageMaker Canvas disponibiliza uma tabela de classificação de modelos candidatos apenas no modo *Standard Build* (2-4 horas de treino). Em modo *Quick Build* (utilizado neste trabalho) esta tabela de classificação não está acessível, o que significa que o utilizador não tem visibilidade sobre quais os algoritmos explorados ou os hiperparâmetros utilizados. O modelo vencedor é apresentado diretamente no separador *Analyze* sem qualquer informação sobre o processo de seleção.

5.4.2.5 Previsões e Deploy

Após o treino, o SageMaker Canvas disponibiliza duas modalidades de previsão, *Single Prediction* e *Batch Prediction*. A *Single Prediction* permite introduzir manualmente os valores de cada atributo e obter imediatamente a previsão com a probabilidade associada a cada classe. A *Batch Prediction* permite carregar um ficheiro CSV com múltiplas amostras e obter as previsões em massa, exportáveis para análise posterior.

A implementação em produção é feita diretamente a partir da interface, sem configuração adicional de infraestrutura. O módulo MLOps permite acompanhar o desempenho em produção ao longo do tempo, identificando desvios entre a distribuição dos dados de treino e os dados reais recebidos.

5.4.2.6 Conclusões

O SageMaker Canvas é a melhor escolha para a prototipagem rápida e para provas de viabilidade, sendo a única plataforma verdadeiramente no-code avaliada. A sua velocidade e acessibilidade são vantagens significativas em contextos onde o tempo de submissão é crítico, ainda que à custa de opacidade total sobre o processo interno em modo de construção rápida.

Vantagens:

- Interface no-code que cobre todo o processo de criação do modelo sem programação ou configuração de infraestrutura;
- Modo *Quick Build* produz um modelo em 2-15 minutos;
- Explicabilidade integrada via SHAP com gráficos por classe acessíveis a utilizadores sem formação técnica.

Desvantagens:

- Opacidade total em *Quick Build*: sem acesso a tabela de classificação, aos algoritmos testados ou aos hiperparâmetros;
- Controlo limitado sobre o processo de AutoML;
- Dependência do ecossistema AWS dificulta a portabilidade;
- Tabela de classificação e detalhes do modelo apenas disponível em *Standard Build* que demora 2-4 horas e produz custos adicionais.

5.4.3 Google Vertex AI AutoML

O Google Vertex AI [10] é a plataforma unificada de Aprendizagem Automática da Google Cloud, que reúne num único ambiente ferramentas de preparação de dados, treino, avaliação, implementação em produção e acompanhamento, com integração nativa com BigQuery, Cloud Storage e Dataflow.

O módulo AutoML Tabular disponibiliza dois modos, o AutoML em *pipelines*, com maior visibilidade do processo mas configuração de permissões mais complexa, e o AutoML Clássico, mais direto e adequado para contas de avaliação gratuita. Em ambos, a plataforma produz automaticamente o pré-processamento, a seleção de algoritmos e a

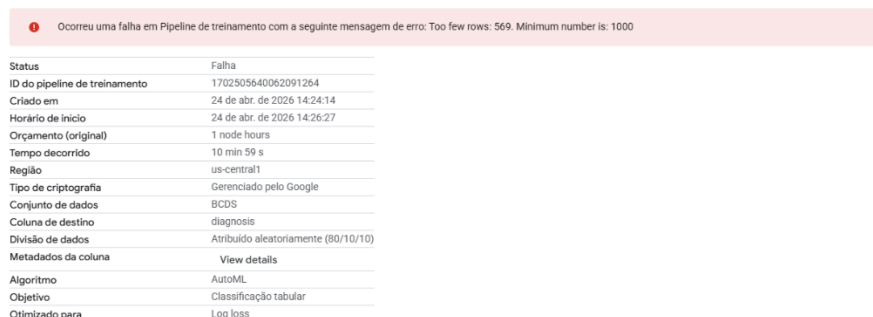
construção do modelo final, fornecendo explicabilidade através do método *Sampled Shapley*.

5.4.3.1 Conjunto de Dados e Importação

5.4.3.1.1 Primeira Tentativa – “Breast Cancer Wisconsin”

O processo iniciou-se com o conjunto de dados “Breast Cancer Wisconsin”. Foi criado um conjunto de dados tabular na secção Conjuntos de dados do Vertex AI, com objetivo de classificação, e carregado o ficheiro `data.csv` a partir do computador local. A coluna `diagnosis` foi definida como variável alvo.

Após a submissão do *pipeline* AutoML, o processo falhou com a mensagem ‘Too few rows: 569. Minimum number is: 1000’ (Figura 11). O Vertex AI AutoML para dados tabulares exige um mínimo de 1000 linhas de treino. Esta limitação obrigou a substituir este conjunto de dados por um com volume suficiente.



Ocorreu uma falha em Pipeline de treinamento com a seguinte mensagem de erro: Too few rows: 569. Minimum number is: 1000

Status	Falha
ID do pipeline de treinamento	1702505640062091264
Criado em	24 de abr. de 2026 14:24:14
Horário de início	24 de abr. de 2026 14:26:27
Orçamento (original)	1 node hours
Tempo decorrido	10 min 59 s
Região	us-central1
Tipo de criptografia	Gerenciado pelo Google
Conjunto de dados	BCDS
Coluna de destino	diagnosis
Divisão de dados	Atribuído aleatoriamente (80/10/10)
Metadados da coluna	View details
Algoritmo	AutoML
Objetivo	Classificação tabular
Otimizado para	Log loss

Figura 11 – Erro ao importar “Breast Cancer Wisconsin”

5.4.3.1.2 Segunda Tentativa – “Adult Income”

Face ao requisito mínimo, foi adotado o conjunto de dados “Adult Income” do UCI ML Repository, que descreve características demográficas e laborais de indivíduos norte-americanos, tendo como variável alvo `income`, o que indica se o rendimento anual é superior ou inferior a 50 000 dólares. Trata-se de um problema de classificação binária com desequilíbrio moderado de classes (cerca de 75% vs. 25%).

Antes do carregamento, foi necessário realizar um pré-processamento em Python, as colunas `workclass`, `occupation` e `native_country` continham valores desconhecidos codificados como `?`, que foram removidos, e os nomes de colunas com hífens foram substituídos por underscores, uma vez que o Vertex AI não os aceita. O pré-processamento foi realizado no notebook `pre-processamento_adults.ipynb`, disponível na pasta Google Vertex AI do GitHub.

5.4.3.2 Configuração do AutoML Job

A exploração percorreu os dois modos de AutoML disponíveis, o AutoML em *pipelines* (com o “Breast Cancer Wisconsin”, que falhou por insuficiência de linhas) e o AutoML Clássico (com o “Adult Income”, que concluiu com sucesso).

O AutoML em *pipelines* segue um processo de cinco passos, como se observa na tabela 10. Durante a configuração deste modo foram necessárias permissões IAM adicionais para a conta de serviço do Vertex AI, nomeadamente os papéis Vertex AI User, Storage Object Admin e Editor.

Passo	Fase	Valor Configurado	Descrição
Passo 1	Detalhes da execução	AutoML Tabular Classification / Regression	<i>Pipeline</i> da galeria de modelos. Nome criado automaticamente
Passo 2	Ambiente de execução	Bucket: cloud-ai-platform-...	Diretório de saída no Cloud Storage. Exigiu configuração de permissões IAM adicionais
Passo 3	Método de treino	Conjunto de dados: “BCW”; Objetivo: Classificação; Target: <i>diagnosis</i>	Coluna alvo definida manualmente. Modelo nomeado BCDS
Passo 4	Opções de treino	15 atributos; Transformação: Automático	Todas as colunas exceto <i>target</i> incluídas. Transformações aplicadas automaticamente pelo AutoML
Passo 5	Computação e preços	Orçamento: 1 node-hour	Parada antecipada ativada. Cobra por tempo de computação consumido

Tabela 10 - Configuração do job AutoML em *pipelines* para o “Breast Cancer Wisconsin”

A configuração do job de AutoML Clássico para o “Adult Income” definiu o tipo de tarefa como classificação, a coluna alvo como `income`, e as transformações como automáticas, como se observa na tabela 11. A plataforma aplicou autonomamente encoding para as variáveis categóricas e normalização para as numéricas. O orçamento de computação foi fixado em 1 node-hour com paragem antecipada ativada, o que significa que o treino termina automaticamente quando o modelo converge, mesmo antes de esgotar o orçamento.

Parâmetro	Valor configurado
Modo AutoML	AutoML Clássico
Tipo de tarefa	Classificação binária ($\leq 50K$ vs $> 50K$)
Conjunto de dados	“Adult Income”
Target column	<i>income</i>
Transformações	Automáticas (encoding, normalização, imputação)
Orçamento de computação	1 node-hour
Parada antecipada	Ativada
Divisão dos dados	80/10/10 (treino/validação/teste)
Duração efetiva	2 horas e 14 minutos

Tabela 11 - Configuração do job AutoML Clássico para o “Adult Income”

5.4.3.3 Resultados do Modelo

O modelo AutoML para o conjunto de dados “Adult Income” foi treinado com sucesso em modo AutoML Clássico e concluído em 2 horas e 14 minutos. A divisão dos dados foi feita aleatoriamente na proporção 80/10/10. As métricas de avaliação foram calculadas com um limite de confiança de 0.5 sobre o conjunto de teste (10% dos dados, correspondendo a aproximadamente 3.000 amostras).

5.4.3.3.1 Curvas de Avaliação e Matriz de Confusão

O separador Avaliar do Vertex AI apresenta três curvas de avaliação interativas, a curva Precisão/Recall (Figura 12), a curva ROC (Figura 13) e o gráfico Precisão/Recall por Limite de Confiança, que neste caso foi de 0.95 (Figura 14). A curva Precisão/Recall é especialmente relevante para este conjunto de dados dado o desequilíbrio de classes. O gráfico de Precisão/Recall por Limite permite ao utilizador seleccionar o threshold de classificação ótimo para o seu contexto específico, observando em tempo real o efeito de cada limiar sobre a precisão e o recall.

Precision-recall curve ?

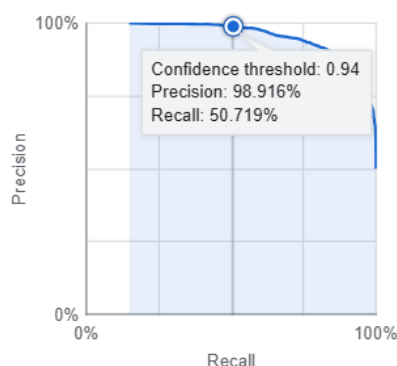


Figura 12 - Curva Precisão/Recall

ROC curve ?

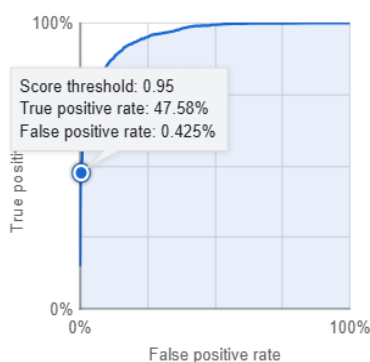


Figura 13 - Curva ROC

Precision-recall by threshold ?

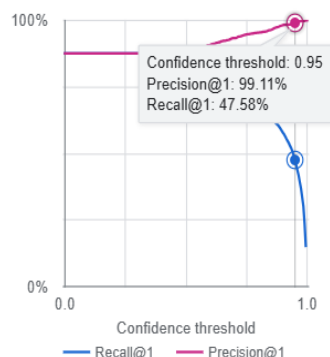


Figura 14 - Gráfico Precisão/Recall

A matriz de confusão (Figura 15) apresenta os resultados em valores absolutos. A classe $\leq 50K$ é classificada corretamente em 1283 casos, com 1009 amostras classificadas como "Dropped" (não classificadas pelo modelo). A classe $>50K$ apresenta mais dificuldade, com 13 casos de falsos negativos e 581 amostras não classificadas ("Dropped").

True label	Predicted label		
	$\leq 50K$	$>50K$	Dropped
$\leq 50K$	1283	0	1009
$>50K$	13	172	581

Figura 15 - Matriz de confusão

5.4.3.3.2 Importância dos Atributos (SHAP)

O Vertex AI AutoML utiliza o método *Sampled Shapley* para estimar a contribuição marginal de cada atributo por amostra, expressa em percentagem. Os resultados revelam que `marital_status`, `occupation` e `education_num` representam aproximadamente 60% do poder preditivo total do modelo (Figura 16).

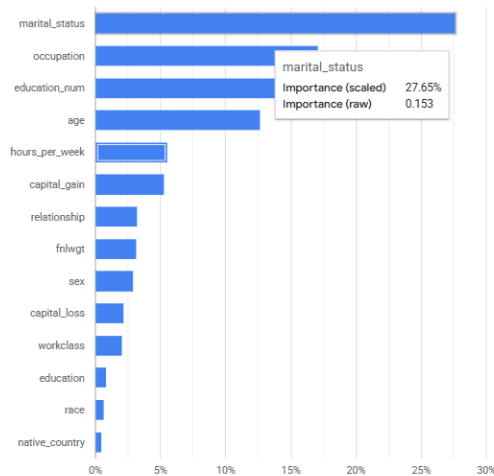


Figura 16 - Importância dos atributos calculada por Sampled Shapley

5.4.3.4 Tabela de Classificação e Transparência do Processo

O Vertex AI AutoML não disponibiliza uma tabela de modelos candidatos, o utilizador vê apenas o modelo final, sem acesso aos algoritmos explorados, aos hiperparâmetros individuais ou ao código criado internamente. O separador Detalhes da versão apresenta os parâmetros de configuração, o método de explicabilidade e o orçamento de computação utilizado.

5.4.3.5 Deploy e Integração

Após o treino, o Vertex AI disponibiliza duas modalidades de inferência, um *endpoint* REST para inferência em tempo real e inferência em lote para previsões em massa sobre conjuntos de dados armazenados no Cloud Storage.

A integração nativa com o ecossistema Google Cloud é uma das principais vantagens da plataforma, pois os conjuntos de dados podem ser importados diretamente do BigQuery ou do Cloud Storage e o Model Monitoring permite configurar alertas de desvio de dados e previsões. Esta integração é especialmente relevante para organizações já integradas no ecossistema Google Cloud.

5.4.3.6 Conclusões

O Google Vertex AI AutoML adequa-se a organizações com maturidade técnica que já operam no ecossistema Google Cloud. A sua limitação de mínimo de 1000 linhas condiciona a aplicabilidade a conjuntos de dados de pequena dimensão, como se verificou neste estágio.

Vantagens:

- Explicabilidade nativa por *Sampled Shapley* disponível imediatamente após o treino;
- Curvas de avaliação interativas e matriz de confusão com percentagens por classe;
- Integração nativa com o ecossistema Google Cloud;
- Dois modos de AutoML tabular adaptáveis ao perfil técnico da equipa.

Desvantagens:

- Requisito mínimo de 1000 linhas;
- Configuração de permissões IAM complexa para o AutoML em *pipelines*;
- Opacidade do processo interno, sem tabela de classificação e sem acesso ao código criado nem aos hiperparâmetros;
- Interface pressupõe familiaridade com conceitos de cloud, sendo menos acessível a utilizadores não técnicos.

5.5 Análise Comparativa das Plataformas Cloud

As três plataformas partilham o mesmo objetivo, automatizar a criação de modelos preditivos sem programação extensiva, mas refletem filosofias distintas sobre o que um utilizador de AutoML necessita.

A diferença mais imediata está na forma como cada plataforma lida com a infraestrutura. O SageMaker Canvas adota uma abordagem onde o utilizador seleciona o conjunto de dados, define a variável alvo e inicia o treino, sem qualquer configuração de recursos. O Azure ML segue a abordagem oposta, exigindo a criação e configuração explícita de um *compute cluster* antes do primeiro treino. O Vertex AI ocupa uma posição intermédia, o utilizador define um orçamento em node-hours sem escolher o tipo de máquina, mas tem de configurar permissões IAM, especialmente no modo AutoML em *pipelines*.

O grau de transparência sobre o processo interno também varia significativamente. O Azure ML é o mais transparente, expõe o código Python criado, os ficheiros JSON de configuração, a tabela completa de candidatos com hiperparâmetros e o painel Ensemble details. O SageMaker Canvas em modo *Quick Build* é o menos transparente, o utilizador vê apenas o modelo final sem informação sobre os algoritmos testados (a tabela de candidatos só está disponível em *Standard Build*). O Vertex AI situa-se no meio, expõe os parâmetros de configuração da tarefa, mas não expõe os candidatos explorados nem os hiperparâmetros do modelo final.

Por último, a gestão dos dados difere entre plataformas. O Canvas é o mais automático, após o carregamento o Data Wrangler cria histogramas, deteta tipos e identifica valores em falta sem intervenção. O Azure ML segue um assistente com diferentes passos onde o utilizador valida os tipos de cada coluna. O Vertex AI é o mais exigente, o “Adult Income” exigiu pré-processamento externo em Python antes do carregamento, e a plataforma impõe um requisito mínimo de 1000 linhas que excluiu o “Breast Cancer Wisconsin”, ao contrário das outras duas plataformas.

Limitação metodológica: a comparação entre plataformas não foi realizada sobre o mesmo conjunto de dados, por imposição do requisito mínimo da plataforma Vertex AI. Esta assimetria impede uma comparação direta de resultados, uma vez que as diferenças de desempenho observadas refletem simultaneamente as capacidades das plataformas e as características dos dados.

5.5.1 Implicações para a Prática de Consultoria

A seleção da plataforma de AutoML adequada é uma decisão estratégica que depende do perfil da equipa, do tempo de entrega, dos custos operacionais e da capacidade de explicar os resultados perante o cliente.

O SageMaker Canvas é a escolha natural para prototipagem rápida e provas de viabilidade onde o cliente quer ver resultados em horas. A ausência de configuração de infraestrutura e o modo *Quick Build* permitem demonstrar um modelo funcional no próprio dia em que os dados são recebidos, e a explicabilidade SHAP é eficaz para audiências não técnicas. A principal limitação é a opacidade total em *Quick Build*, se o cliente ou um auditor questionar como o modelo foi construído, a plataforma não fornece resposta.

O Azure ML posiciona-se para projetos onde a auditabilidade e a integração com MLOps são requisitos, particularmente em setores regulados como banca, seguros ou saúde. O acesso ao código criado e à tabela detalhada de candidatos permite documentar e

justificar cada etapa. O custo é a complexidade operacional, tarefas de 6 ou mais horas não permitem iteração rápida e a configuração de *compute clusters* pressupõe perfis técnicos.

O Vertex AI adequa-se a organizações com maturidade técnica no ecossistema Google Cloud, onde a integração nativa com BigQuery e Cloud Storage elimina movimentações de dados desnecessárias. O requisito mínimo de 1000 linhas e a necessidade de pré-processamento externo limitam a aplicabilidade a projetos com dados de dimensão pequena e equipas familiarizadas com GCP.

Uma consideração transversal é o *vendor lock-in*, as três plataformas operam em ecossistemas fechados, e a regra prática em consultoria é alinhar a escolha com o ecossistema já adotado pelo cliente, pois introduzir uma plataforma de um fornecedor diferente cria fricção desnecessária.

5.5.2 Síntese Crítica e SWOT Comparativa

A exploração das três plataformas permitiu verificar que o conceito não designa uma tecnologia uniforme, mas sim um conjunto de abordagens com diferentes equilíbrios entre acessibilidade, transparência e controlo. As tabelas 12, 13 e 14 apresentam a análise SWOT individual de cada plataforma.

Azure ML AutoML

Forças	Fraquezas
- Transparência total; - Tabela de classificação completa; - Painel Ensemble Details - Monitorização de recursos e comparação de tarefas.	- Complexidade de configuração de <i>compute clusters</i>; - Tempo de treino elevado; - Custos de VM por hora acumulam em projetos longos;
Oportunidades	Ameaças
- Projetos regulados que exigem auditabilidade; - Clientes no ecossistema Microsoft; - Integração com MLOps em produção;	- Complexidade pode afastar utilizadores; - Custo elevado em conjuntos de dados grandes; - <i>Timeout</i> sem conclusão total da pesquisa.

Tabela 12 - Análise SWOT do Azure ML AutoML

SageMaker Canvas

Forças	Fraquezas
- Interface no-code acessível a qualquer perfil; - <i>Quick Build</i> em 2-15 minutos; - Explicabilidade SHAP visual.	- Opacidade total do processo em <i>Quick Build</i>; - Tabela de classificação apenas em <i>Standard Build</i>; - Controlo limitado sobre algoritmos e hiperparâmetros.
Oportunidades	Ameaças
- Prototipagem rápida e provas de viabilidade; - Democratização do ML em equipas não técnicas; - Integração com ecossistema AWS já existente no cliente.	- Dificuldade de auditoria em contextos regulados; - Custos opacos de computação em <i>Standard Build</i>; - Dependência da disponibilidade da AWS.

Tabela 13 - Análise SWOT do SageMaker Canvas

Google Vertex AI

Forças	Fraquezas
- Integração nativa com BigQuery e Cloud Storage; - Curvas de avaliação interativas; - <i>Sampled Shapley</i> disponível sem configuração; - Dois modos de AutoML tabular.	- Requisito mínimo de 1000 linhas; - Configuração IAM complexa no modo AutoML em <i>pipelines</i>; - Sem tabela de classificação; - Pré-processamento externo em alguns casos.
Oportunidades	Ameaças
- Clientes com dados já no Google Cloud (BigQuery); - Conjuntos de dados volumosos.	- Barreira de entrada elevada para utilizadores não técnicos; - Instabilidade em contas de avaliação gratuita; - <i>Vendor lock-in</i> no ecossistema Google Cloud.

Tabela 14 - Análise SWOT do Google Vertex AI

A exploração revelou também que a promessa de no-code das plataformas de AutoML tem limites práticos importantes. AutoML não elimina a necessidade de literacia em Aprendizagem Automática, reduz a barreira de entrada mas não a remove completamente.

5.6 Comparação entre Ferramentas Open-Source e Plataformas Cloud

A exploração prática das quatro ferramentas e três plataformas avaliadas neste estágio permite agora formular uma análise comparativa transversal que vai além das métricas de desempenho individuais. Esta secção sistematiza as diferenças estruturais entre os dois paradigmas.

5.6.1 Acessibilidade e Curva de Aprendizagem

As ferramentas open-source exigem, sem exceção, conhecimento de Python e familiaridade com ambientes de desenvolvimento. O TPOT, o AutoGluon e o H2O AutoML são acessíveis a qualquer cientista de dados com experiência em Python e scikit-learn, mas cada ferramenta tem as suas convenções de API e parâmetros de configuração que requerem leitura de documentação antes de se atingir utilização produtiva. O AdaNet representa um caso extremo, com uma API TensorFlow de baixo nível que pressupõe conhecimento sólido de aprendizagem profunda.

As plataformas cloud situam-se num conjunto mais alargado. O SageMaker Canvas representa o extremo mais acessível, tendo uma interface no-code que permite obter um modelo funcional em menos de uma hora, desde o carregamento dos dados até à análise dos resultados. O Vertex AI e o Azure ML ocupam uma posição intermédia, oferecem interfaces gráficas que simplificam a configuração, mas requerem conhecimento básico de Aprendizagem Automática. A promessa de no-code tem, portanto, limites práticos que este estágio confirmou. O Vertex AI exigiu pré-processamento externo em Python, e o Azure ML requereu configuração de infraestrutura antes do primeiro treino.

5.6.2 Controlo, Transparência e Auditabilidade

As ferramentas open-source oferecem, por definição, acesso total ao código, aos *pipelines* criados, aos hiperparâmetros explorados e aos registos de execução, um requisito essencial em contextos onde a auditabilidade é exigida por reguladores ou por clientes com políticas rigorosas. O TPOT exporta o *pipeline* final em Python puro, o AutoGluon e o H2O disponibilizam modelos exportáveis em vários formatos. No AdaNet, embora o código seja acessível, os resultados produzidos são orientados para diagnóstico interno do TensorFlow, dificultando a interpretação e auditoria por parte de terceiros.

No segmento cloud, o Azure ML destaca-se como a plataforma mais transparente, disponibilizando o código criado, os registos completos e os hiperparâmetros de cada candidato. O Vertex AI disponibiliza curvas de avaliação interativas e explicabilidade por *Sampled Shapley*, mas não apresenta a tabela de candidatos explorados nem o código interno. O SageMaker Canvas em modo *Quick Build* é o menos transparente, o processo de seleção de algoritmos é completamente opaco e não existe forma de auditar as decisões tomadas internamente.

5.6.3 Desempenho Preditivo e Qualidade dos Modelos

A comparação direta de desempenho é metodologicamente limitada pela impossibilidade de aplicar todas as ferramentas ao mesmo conjunto de dados nas mesmas condições, como descrito na subsecção 5.5.

No segmento open-source, o AutoGluon e o H2O AutoML produziram consistentemente os melhores resultados em dados de complexidade moderada, beneficiando das suas capacidades de comité e da diversidade de algoritmos explorados. O TPOT demonstrou robustez em dados simples, atingindo resultados comparáveis com *pipelines* mais compactos e interpretáveis, e o AdaNet não revelou vantagens de desempenho nos conjuntos de dados tabulares testados.

As plataformas cloud produziram modelos competitivos, o SageMaker Canvas atingiu um AUC-ROC de 0,99 com apenas 3 erros de classificação em 115 amostras no “Breast Cancer Wisconsin”, o Azure ML gerou um `VotingEnsemble` com métricas igualmente elevadas no mesmo conjunto de dados, e o Vertex AI demonstrou desempenho sólido no “Adult Income”. Estes resultados confirmam que as plataformas cloud produzem modelos de qualidade comparável às ferramentas open-source, com menor intervenção manual.

5.6.4 Custo e Escalabilidade

As ferramentas open-source são gratuitas, limitando-se ao hardware disponível, sendo atrativas em fases de investigação ou com orçamento limitado. O H2O AutoML é a exceção parcial, suportando execução distribuída em clusters para grande escala.

As plataformas cloud eliminam as restrições de hardware mas introduzem custos por hora que podem ser significativos em projetos longos. O Azure ML AutoML, com jobs de 6+ horas, representa um custo acumulado não negligenciável. O SageMaker Canvas em *Quick Build* destaca-se pela eficiência, com tempos entre 2 e 15 minutos sem configuração de infraestrutura. O Vertex AI cobra por node-hours com orçamento definido pelo

utilizador, oferecendo um modelo de custo mais previsível. A escalabilidade das plataformas cloud é virtualmente ilimitada, tornando-as preferíveis para projetos empresariais de grande escala.

5.6.5 Síntese Comparativa

A análise comparativa das duas categorias de ferramentas evidencia que não existe uma solução universalmente superior, a escolha ótima depende do perfil da equipa, dos requisitos do projeto, das restrições orçamentais e do ecossistema tecnológico já adotado. A tabela 15 sintetiza as principais dimensões de comparação entre os dois paradigmas, com base na experiência prática deste estágio.

Dimensão	Ferramentas Open-Source	Plataformas Cloud
Perfil de utilizador	Data scientist / engenheiro de ML com Python	Analista de negócio a data scientist, variável por plataforma
Custo de acesso	Gratuito	Pay-per-use (custo por hora de computação)
Transparência do processo	Total	Variável: total (Azure ML) a opaca (Canvas <i>Quick Build</i>)
Escalabilidade	Limitada pelo hardware local (exceto H2O em cluster)	Virtualmente ilimitada (gerida pelo fornecedor cloud)
Vendor lock-in	Ausente	Elevado
Adequação a projetos regulados	Alta	Variável: alta (Azure ML) a baixa (Canvas <i>Quick Build</i>)

Tabela 15 – Comparação estrutural entre ferramentas open-source e plataformas cloud de AutoML

As ferramentas open-source e as plataformas cloud não são concorrentes diretos, mas complementares. Cada uma responde a um conjunto distinto de requisitos operacionais, técnicos e organizacionais. A competência de um profissional de dados não reside na escolha exclusiva de um paradigma, mas na capacidade de selecionar a ferramenta certa para cada contexto.

6 Avaliação crítica

O desenvolvimento deste projeto de exploração e teste de técnicas de AutoML proporcionou conhecimentos valiosos, tanto técnicos como metodológicos, que transcendem a simples interpretação de resultados quantitativos. Esta secção reflete sobre as aprendizagens mais significativas e sobre o que faria de forma diferente se fosse recomeçar.

6.1 Aprendizagens Técnicas

A primeira aprendizagem foi a distância entre expectativas teóricas e resultados práticos. Comecei este trabalho convicta de que ferramentas mais sofisticadas como o AutoGluon e o H2O demonstrariam superioridade generalizada sobre o TPOT. No “Iris”, as diferenças foram marginais, dado que qualquer algoritmo razoável tem bom desempenho num problema simples. No “Breast Cancer Wisconsin”, o AutoGluon e o H2O beneficiaram efetivamente da sua sofisticação, enquanto o AdaNet, apesar da arquitetura TensorFlow avançada, não demonstrou qualquer vantagem. Ficou claro que a escolha da ferramenta depende do contexto e não de uma hierarquia de sofisticação universal.

Um momento particularmente revelador foi a exploração dos mecanismos de configuração. Esperava que todas as ferramentas suportassem ficheiros externos de forma nativa, o que não se verificou. O TPOT havia descontinuado o `config_dict`, e o AutoGluon e o H2O suportavam configuração via JSON, mas apenas através de código Python intermediário. Esta dimensão de maturidade é frequentemente ignorada nos estudos comparativos publicados.

Nas plataformas cloud, a experiência com o Vertex AI foi igualmente instrutiva, o requisito mínimo de 1000 linhas inviabilizou o uso do “Breast Cancer Wisconsin”, que correu sem problemas no SageMaker Canvas e no Azure AutoML. Estas limitações práticas raramente estão documentadas e só se tornam visíveis na experimentação direta.

6.2 Lições Metodológicas

A maior surpresa metodológica não foi a complexidade de testar múltiplas ferramentas, mas sim a necessidade de ampliar o conjunto de métricas ao longo do projeto. O que parecia inicialmente adequado (exatidão e F1-Score) revelou limitações quando confrontado com dados reais, obrigando à inclusão de AUC-ROC, AUPRC, matrizes de confusão detalhadas e análise SHAP.

Um desafio mais exigente foi construir uma comparação justa entre ferramentas com filosofias tão diferentes. Comparar o TPOT que otimiza um único *pipeline*, com o AutoGluon, que constrói comités de dezenas de modelos, obriga a clarificar o que se está efetivamente a comparar, o melhor desempenho possível ou a relação entre desempenho, custo computacional e interpretabilidade.

Se pudesse recomeçar, teria dedicado mais tempo ao desenho da experimentação antes de a iniciar. A necessidade de ajustar a metodologia a meio do processo enriqueceu o projeto, mas também originou trabalho que uma fase de planeamento mais cuidada teria evitado.

6.3 Dimensão Humana e Comunicativa

A comunicação dos resultados, principalmente com o meu orientador Nuno Miranda (Senior Manager) e co-orientador Miguel Silvério (Data Scientist), revelou-se um exercício de equilíbrio entre rigor técnico e clareza expositiva. O maior desafio não foi explicar conceitos técnicos a especialistas, mas sim organizar e apresentar os dados de forma a que as conclusões emergissem naturalmente.

Este projeto ensinou-me que a comunicação eficaz de resultados complexos não é meramente uma questão de simplificação, mas de estruturação cuidadosa. As tabelas comparativas, os gráficos de desempenho e as matrizes de confusão não são apenas complementos estéticos, mas ferramentas fundamentais de comunicação que podem obscurecer ou iluminar as conclusões, dependendo de como são concebidas.

A colaboração com especialistas evidenciou também a importância de um vocabulário comum. Termos como "desempenho" ou "adequação" têm significados diferentes em contextos técnicos e aplicados, originando expectativas desalinhadas que exigiram ajustamento constante. É uma lição que pretendo aplicar em projetos futuros, definir critérios de sucesso explícitos e partilhados por todos os intervenientes antes de começar.

6.4 Propostas para Trabalhos Futuros

A avaliação realizada neste estágio ficou naturalmente limitada pela dimensão e diversidade dos conjuntos de dados utilizados. Uma extensão natural seria aplicar as mesmas ferramentas a problemas de regressão, séries temporais e dados com maior dimensionalidade, obtendo conclusões mais generalizáveis.

Ficou por explorar a combinação de ferramentas open-source com plataformas cloud, por exemplo usar o AutoGluon para seleção de algoritmos e o Azure ML para colocação em produção e monitorização, o que poderia oferecer o melhor de ambos os paradigmas.

Por último, seria interessante explorar o papel dos modelos de linguagem na automatização da configuração de ferramentas de AutoML, uma direção ainda pouco explorada na literatura e com potencial relevância para o contexto de consultoria.

6.5 Reflexões Finais

Este projeto modificou completamente a minha compreensão sobre avaliação de ferramentas de Aprendizagem Automática. O que inicialmente parecia um exercício primariamente técnico revelou-se uma interseção complexa entre engenharia, metodologia científica, design de experiência e comunicação estratégica.

A principal lição foi a necessidade de me libertar de convicções prévias. Ferramentas que dominam determinados benchmarks frequentemente falham em casos de uso específicos, enquanto soluções aparentemente mais simples demonstram robustez surpreendente em aplicações práticas como o TPOT demonstrou no “Iris”, e o SageMaker Canvas demonstrou na classificação do “Breast Cancer Wisconsin”. Esta observação é particularmente relevante para o contexto empresarial da KPMG, onde a rapidez de prototipagem e a facilidade de uso são muitas vezes tão importantes quanto o desempenho absoluto do modelo.

Em última análise, este trabalho reforçou a minha convicção de que estamos apenas a começar a desenvolver metodologias adequadas para avaliar sistemas de *Automated Machine Learning*. As ferramentas e métricas atuais abrangem apenas dimensões limitadas de um fenómeno multifacetado, sugerindo que o campo de avaliação de ferramentas AutoML permanecerá tão dinâmico quanto as próprias ferramentas que avalia. A experiência na KPMG constituiu um ponto de partida sólido e estimulante para uma carreira na área de dados e Aprendizagem Automática.

Glossário

HPO - Otimização automática dos parâmetros de configuração de um modelo.

MLOps Automatizado - Conjunto de práticas que automatizam a implementação, monitorização e manutenção de modelos em produção.

APIs - Application Programming Interfaces: interfaces que permitem a comunicação entre sistemas de software.

Pipeline - Sequência estruturada de etapas de processamento e modelação de dados.

Comités - Conjuntos de múltiplos modelos combinados para produzir uma previsão final mais robusta.

Neural Architecture Search - Técnica que automatiza o design de arquiteturas de redes neurais.

Modus operandi - Modo de funcionamento.

Stacking multi-nível - Técnica de combinação de modelos em camadas hierárquicas para melhorar o desempenho preditivo.

PDPs — Partial Dependence Plots: gráficos que mostram o efeito marginal de cada variável nas previsões do modelo.

TensorFlow — Framework open-source da Google para desenvolvimento e treino de modelos de aprendizagem profunda.

API estimator — Interface do TensorFlow para definição e treino de modelos, atualmente em processo de descontinuação.

Vendor lock-in — Dependência excessiva de um fornecedor tecnológico específico, dificultando a migração para alternativas.

Referências

- [1] Alcobaça, E., & de Carvalho, A. C. P. L. F. (2026). *A literature review on automated machine learning*. Artificial Intelligence Review, 59, Article 5 <https://link.springer.com/article/10.1007/s10462-025-11397-2>
- [2] Tătaru, G.-C., Cosac, A., Ioanăș, I., Florescu, M.-S., Orzan, M., Delcea, C., & Cotfas, L.-A. (2025). *Understanding the rise of automated machine learning: A global overview and topic analysis*. Information, 16(11), Article 994 [Understanding the Rise of Automated Machine Learning: A Global Overview and Topic Analysis](#)
- [3] He, X., Zhao, K., e Chu, X. (2021). AutoML: A survey of the *state-of-the-art*. *Knowledge-Based Systems*, 212, 106622. [AutoML: A survey of the state-of-the-art - ScienceDirect](#)
- [4] Epistasis Lab. (n.d.). *TPOT*. TPOT documentation. Retrieved April 16, 2026, from [TPOT - AutoML](#)
- [5] AutoGluon. (2026). *AutoGluon 1.5.0 documentation*. Retrieved March 28, 2026, from <https://auto.gluon.ai/stable/index.html>
- [6] H2O.ai. (2017, June 6). *AutoML: Automatic machine learning*. H2O 3.12.0.1 documentation [AutoML: Automatic Machine Learning — H2O 3.12.0.1 documentation](#)
- [7] AdaNet Authors. (2018). *adanet 0.9.0 documentation*. Read the Docs. [adanet — adanet 0.9.0 documentation](#)
- [8] Microsoft. (2025, August 29). *Set up AutoML with Python (v2) - Azure Machine Learning*. Microsoft Learn. [Set up AutoML with Python \(v2\) - Azure Machine Learning | Microsoft Learn](#)
- [9] Amazon Web Services. (n.d.). *Free cloud computing services - AWS Free Tier*. Retrieved May 2, 2026, from [Serviços de computação em nuvem gratuitos - Nível gratuito da AWS](#)
- [10] Google Cloud. (n.d.). *Gemini Enterprise Agent Platform*. Retrieved May 7, 2026, from [Gemini Enterprise Agent Platform \(antiga Vertex AI\) | Google Cloud](#)